

Agentforce Observability & Debugging Query Cookbook

Version 3 · 19 August 2026 · 65 ready-to-run Data Cloud queries for tracing, costing and improving Agentforce agents — grouped by the question you are trying to answer, each with what it answers, when to run it, and how to read the result.

This is a query notebook. Copy a recipe, swap in your session id and time window, and run it in the Data Cloud Query editor or the Query API. The groups follow the questions you actually ask: *is data flowing? what happened in this session? why was it slow? what did it cost? what errored? was the answer any good? did my change help?*

New to this? Read three pages, then start querying. **Section 1** shows you where to actually type these queries — the Query Editor, the Query API, Apex, or the CLI — and what has to be switched on in the org first. **Section 2** is the seven rules that cause most bad results; they take two minutes and save hours. Then **Group 0** gets you the four values every other recipe needs. After that, jump to whichever group matches your question.

If a query returns nothing, or will not run at all, go to **Troubleshooting** near the back before changing any configuration — the usual causes are a wrong data space, a session that is simply too new, or a filter you have not set, not a broken org.

The groups at a glance

Group	Answers	Needs
0 Scope before you query	Which data space, which agent, which window — the four values every other recipe needs.	—
A Setup checks	Is data flowing, or is the session just too new for these tables?	—
B Traceability	What happened in this session: transcript, turns, routing, steps.	Session Tracing
C Time from timestamps	Why did it feel slow? Turn and step durations, action latency, time to first response.	Session Tracing
T Precise timing with spans	Exact nanosecond breakdown, percentiles, and where the time actually goes.	Platform Tracing
D Tokens & cost	What did it consume — per call, per turn, per template, per session.	—
E Errors & adherence	What genuinely failed, and where the model ignored its instructions.	Session Tracing
Q Quality	Was the answer any good? Quality tags, trust-layer detectors, RAG grounding, real user feedback.	Some need Optimization
X Deep inspection	Action arguments, tool-call payloads, variable state — where most wrong answers actually come from.	Mixed
R Baseline & regression	Did my change help? Two versions, two windows, compared.	Mixed
P Playbooks	Seven query chains for the situations you actually face, starting with the fresh-session case.	—

Contents

What Every AI Engineer Needs to Know First	5
Section 1 — How and where to run these queries	7
Section 2 — Read before you run	10
Group 0 — Scope before you query	11
0a. Find the right Data Space	11
0b. Which agents ran, and how much	11
0c. The agent-scoping join	12
0d. How fresh is this session?	12
Group A — Setup checks: is data flowing, or is it just early?	13
A1. Do recent sessions exist?	13
A2. Is token / gateway data flowing?	13
A3. Do platform telemetry spans exist?	14
Group B — Traceability: what happened in this session?	15
B1. Find your session (newest first)	15
B2. All messages in a session (what was said)	15
B3. List the turns in a session	16
B4. List the steps inside one turn	16
B5. Count turns and steps per session	16
B6. What end types does this org actually record?	17
B7. Sessions that ended badly	17
B8. Routing distribution and the unrouted rate	18
Group C — Time, from the step timestamps	19
C1. Slowest turns in a session	19
C2. Turns that never ended	19
C3. Per-step durations for one turn	20
C4. Turn shape: how many of each step type	20
C5. Slowest turns per subagent (one agent, one day)	21
C6. Average turn duration by subagent	21
C7. Action latency leaderboard	22

C8. Time to first response	22
Group T — Precise timing with telemetry spans	24
T1. Slowest operation types, fleet-wide	24
T2. Get the trace id for a turn	24
T3. Every span for one trace	25
T4. Recent error spans, with the reason	25
T5. Slowest LLM spans only	26
T6. Where does latency live — planner, action, or retrieval?	26
T7. Latency percentiles, not averages	26
T8. Unaccounted time inside a trace	27
Group D — Tokens & cost: what did it consume?	28
D1. Every model call for a session	28
D2. Tokens per user turn	28
D3. Total tokens for a session	29
D4. Most expensive requests recently	29
D5. Tokens rolled up by model	30
D6. Exact prompt and response for a step	30
D7. Tokens by feature	31
D8. Tokens by prompt template	31
D9. Prompt vs completion ratio by template	32
D10. Token growth across a conversation	32
D11. Retries and duplicate calls	33
D12. Cost per resolved conversation	33
D13. Sampling-parameter audit	34
D14. Prompt size distribution	34
Group E — Errors & adherence: what actually went wrong?	35
E1. Every genuinely errored step	35
E2. Low instruction-adherence turns	35
E3. Connect a failing step to its span	36
Group Q — Quality: was the answer any good?	37
Q1. Session-level quality scores with reasons	37
Q2. Per-moment quality, and turns-per-intent efficiency	38
Q3. Lowest-scoring sessions	38

Q4. Trust-layer detectors: toxicity, PII, prompt defense, task resolution	39
Q5. Was toxicity detected on any generation?	39
Q6. What did users actually say about the answers?	40
Q7. RAG grounding: faithfulness, relevancy, precision	40
Group X — Deep inspection: the fields nobody looks at	42
X1. What went into and out of an action	42
X2. What the planner tried to call	42
X3. Variable state through the turn	43
X4. Did the user have to ask twice?	43
X5. Did this conversation start somewhere else?	44
X6. PII and safety posture	44
Group R — Baseline & regression: did my change help?	45
R1. Tokens per turn, by agent version	45
R2. Latency percentiles, week over week	45
R3. Quality distribution, two windows	46
R4. Daily trend for one subagent	46
Group P — Scenario playbooks: chain the queries	47
P0 · 'This just happened and I need to know now'	47
P1 · 'This turn was slow'	47
P2 · 'This conversation was expensive'	47
P3 · 'The agent gave a wrong answer'	47
P4 · 'Something is failing'	48
P5 · 'Is my knowledge base the problem?'	48
P6 · 'Did my change actually help?'	48
Reading guide — trace → cause → fix	49
Troubleshooting — when the queries will not run	51
Further reading	54
Appendix — Conflicts to settle in your own org	56

What Every AI Engineer Needs to Know First

The vocabulary, data model, and field conventions these queries assume. Read this page before your first query — it will save you from the mistakes that waste the most time.

The data model

Term	What it means
DMO — Data Model Object	The queryable table, suffix <code>__d__lm</code> . Everything in this notebook queries DMOs.
DLO — Data Lake Object	The raw ingested layer beneath a DMO, suffix <code>__d__ll</code> , with different field naming (camelCase, no <code>ssot__</code> prefix). A separate query surface — never copy field names between the two.
STDM — Session Trace Data Model	The Agentforce tracing DMOs: Session, Participant, Interaction, Message, Step. The backbone of Groups A through E.
Data space	A partition within Data Cloud. Every query runs against exactly one. Choosing the wrong one is the most common cause of an empty result.
Session	One conversation. PK <code>ssot__Id__c</code> — the id you paste into most recipes. Carries channel, start/end, end type, and a link to a prior session.
Participant	A party in a session, role USER or AGENT. The only reliable carrier of the agent API name — it is not on Session.
Interaction / turn	One exchange. Type TURN or SESSION_END. Carries the topic that handled it.
Step	One internal action inside a turn. Five types: TOPIC_STEP, LLM_STEP, ACTION_STEP, TRUST_GUARDRAILS_STEP, SESSION_END. No session FK — reach it through Interaction.
Moment	One user <i>intent</i> , which may span several turns. Requires Agent Optimization. Turns-per-moment is an efficiency signal (Q2).
Topic / subagent	<code>ssot__TopicApiName__c</code> — which part of the agent handled the turn. Null or NOT_SET means no topic was recorded, which in practice usually means the planner could not decide which subagent owned the utterance (B8).
Planner	The reasoning engine that decides routing and tool calls. Its LLM calls carry <code>feature__c</code> = 'plannerservice' — the filter that separates agent traffic from everything else in the org.

The LLM audit chain

Term	What it means
Gateway request	One call to a model, on <code>GenAIGatewayRequest__d__lm</code> . Carries the full prompt, token counts, model, template and sampling parameters. The richest and fastest-landing table you have.
Generation	The model's response, on <code>GenAIGeneration__d__lm</code> . Reached forward from an LLM_STEP via <code>ssot__GenerationId__c</code> — and only from those.
Generation group	<code>generationGroupId__c</code> groups every model call belonging to one user-visible turn. The unit for per-turn token maths.
Prompt / completion tokens	Input versus output. The <i>ratio</i> matters more than the total: input-heavy means context bloat, output-heavy means the model is writing too much, and the fixes are opposite (D9).
Prompt template	<code>promptTemplateDevName__c</code> , e.g. <code>AiCopilot__ReactTopicPrompt</code> . The closest available proxy for "which part of the agent" when attributing cost.

Term	What it means
Trust Layer	Salesforce's safety scoring — toxicity, PII, prompt defense, instruction adherence and task resolution. Surfaces as GenAIContentQuality and GenAIContentCategory rows (Q4, Q5). Needs Einstein Audit & Feedback enabled — but <i>not</i> the Agent Optimization add-on, which is what makes it the accessible quality signal.
Instruction adherence	Did the model follow its instructions? HIGH or LOW, recorded on a TRUST_GUARDRAILS_STEP (E2).
Task resolution	Did the agent actually solve the problem? A trust-layer detector, and the closest thing to a direct success measure the platform gives you (Q4).

Tracing, quality and conventions

Term	What it means
Span / trace	OpenTelemetry-style timing records from Agent Platform Tracing. A trace is one turn; spans are the nested operations inside it. The real trace id usually lives inside ssot__AttributeText__c, not in the trace-id column (T2).
Faithfulness	Was the answer grounded in what was retrieved? Below 0.8 is hallucination risk.
Answer relevancy	Did the answer address the question? Below 0.7 is the published cut-off for flagging a subagent. Read alongside context precision — the two together are what indicate a knowledge gap.
Context precision	Did the retriever return relevant chunks? Low points at the retrieval side — the knowledge content, its chunking, or the filters — rather than at the model.
Quality tag	An LLM-assigned 1–5 score with a written rationale. 5 = High, 3–4 = Medium, 2 = Low, 1 = Very Low. Stored as a <i>string</i> , so cast before averaging.
Materialization	How long a DMO takes to appear after the conversation. Minutes for Session, Participant and Gateway; hours to days for Interaction, Step, Message, Generation and Moment. The single most important operational fact here.
NOT_SET	Data Cloud's "empty" sentinel — the column is not null. Its siblings are UNSET_VALUE and the empty string, and guardrail steps add the literal None.
"no_session"	The sentinel on gateway rows with no owning session — Prompt Builder previews, eval harnesses, test runs. Quotes are part of the stored value. Exclude it from cost roll-ups.
VERIFIED / DERIVED / UNCONFIRMED	This notebook's confidence labels: confirmed against something that runs; our own maths on real fields; plausible but never run live. See Section 2.

Section 1 — How and where to run these queries

These are Data 360 SQL queries against Data Cloud, not SOQL and not Apex. There are four places you can run them; the Query Editor is the one to start with. Everything in this section is setup you do once.

Data Cloud SQL is its own language. It is ANSI-style SQL — closer to PostgreSQL than to anything else in Salesforce. SOQL habits will mislead you: there are real JOINS, real GROUP BY, real window functions, and no relationship-dot syntax. Table names are the DMO API names, and it is good practice to double-quote them (FROM "ssot__AIAgentSession__d1m") though the recipes here read fine without.

Before anything: what has to be switched on

What	Where	Why you care
Data Cloud provisioned on the org	Nothing to query without it	Every table in this notebook is a Data Cloud DMO. No Data Cloud, no data.
Agentforce Session Tracing and Data Model	Setup → Einstein Audit, Analytics, and Monitoring Setup	Turns on the five STDM tables — Session, Participant , Interaction, Message and Step. Almost everything depends on it , including the Participant join that scopes fleet-wide recipes to one agent. Only the gateway-only recipes (A2, D1–D14, X2, X6) work without it.
Agentforce Optimization	Same Setup page	Adds the Moment and quality-tag DMOs. Q1, Q2, Q3 and R3 depend on it; everything else works without.
Agentforce Platform Tracing	Same Setup page, its own toggle — and it requires Session Tracing to be on first . Deploying via metadata instead: Agentforce PlatformTracing.settings-meta.xml (enableAgentforcePlatformTracing), API v68.0+, plus the provisioned PlatformObservability org permission	Turns on telemetry spans. A3, the whole T group, and E3 depend on it. Collection runs every five minutes, so a very recent turn may have no spans yet.
Permission sets on your user	Data Cloud User; plus Access Agentforce Optimization and a Tableau Next consumer or analyst set if you want the Optimization surfaces	Without Data Cloud User the Query Editor will not appear at all. The Optimization surfaces additionally need API Enabled on the profile.

Tracing is not retroactive. Analytics and traces only cover conversations that happen **after** you enable the Session Tracing Data Model. Turning it on today gives you nothing about yesterday's incident — so if you are reading this because something went wrong, check the enablement date before concluding the data is missing.

Path A — The Query Editor (start here)

1. Open the Data Cloud app.

App Launcher → **Data Cloud**. If it is not there, you are missing the Data Cloud User permission set or Data Cloud is not provisioned — see Troubleshooting T9.

2. Go to the Query Editor tab.

Sometimes shown alongside **Data Explorer**. Data Explorer browses a single object; the Query Editor is where you paste SQL.

3. Pick the data space.

There is a data-space selector on the editor. This is the single most common cause of "my query returns nothing" — the default space is often not the one your agent writes to. Recipe **0a** shows how to list the real names.

4. Paste a recipe and replace the placeholders.

Everything in angle brackets — `<SESSION_ID>`, `<AGENT_API_NAME>`, `<ISO_CUTOFF>` — has to be substituted. Nothing in this notebook runs as-is.

5. Run, and read the row count first.

Zero rows is information, not failure. Take it to Troubleshooting rather than to the setup screens.

Computing the `<ISO_CUTOFF>` value. Most recipes filter time with a literal instead of date arithmetic, because the STDm timestamp columns are strings (Rule 1). You need a UTC ISO-8601 string for the start of your window:

- **Seven days ago:** in a terminal, `date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%S.000Z` (GNU) or `date -u -v -7d +%Y-%m-%dT%H:%M:%S.000Z` (macOS).
- **In Apex:** `Datetime.now().addDays(-7).formatGmt('yyyy-MM-dd\T\HH:mm:ss.SSS\Z\')`
- **By hand:** `'2026-08-12T00:00:00.000Z'` is a perfectly good cutoff. Precision rarely matters; being in UTC does.

Path B — The Query API (for scripting and repeat runs)

Use this when you want to run a recipe on a schedule, feed results into a spreadsheet, or diff two windows without re-typing anything. You need a Data Cloud OAuth token and your tenant-specific endpoint — both come out of the token exchange, so do not try to guess the host.

```
POST https://{TSE}.360a.salesforce.com/api/v2/query

Headers:
  Authorization: Bearer {cdpAuthToken}
  Content-Type: application/json

Body:
  { "sql": "SELECT ssot__Id__c FROM ssot__AIAgentSession__dml LIMIT 5" }

Response:
  { "data": [ ... ],           -- the rows
    "rowCount": 5,
    "queryId": "...",
    "done": true,              -- false means more batches to fetch
    "nextBatchId": "...",      -- pass this to get the next batch
    "metadata": { "<column>": { "type": ..., "placeInOrder": ... } },
    "startTime": "...", "endTime": "..." }
```

- **Pagination.** When `done` is `false`, fetch the next batch with `nextBatchId`.
- **Two clocks are running.** You must fetch the complete result set within **one hour**, and no more than **three minutes** may pass between batch calls. A slow consumer loses the result set.
- **Column order.** Rows come back positionally; use the metadata block's `placeInOrder` rather than assuming your `SELECT` order survived.
- The token is a Data Cloud token, not a plain Salesforce session id — it is exchanged from one. If you are wiring this up permanently, do it through a Named Credential rather than handling tokens yourself.

Path C — Apex (when you want it inside the org)

The same queries run from anonymous Apex, a Flow, or an invocable action. This is the path the `AgentforceOptimizeService` helper class uses, and the one to reach for when the result needs to drive something

else in the org.

```
ConnectApi.CdpQueryInput inp = new ConnectApi.CdpQueryInput();
inp.sql = 'SELECT ssot__Id__c, ssot__StartTimestamp__c '
        + 'FROM ssot__AIAgentSession__dlm '
        + 'WHERE ssot__StartTimestamp__c >= \'2026-08-12T00:00:00.000Z\' '
        + 'LIMIT 10';

ConnectApi.CdpQueryOutputV2 result =
    ConnectApi.CdpQuery.queryAnsiSqlV2(inp, 'YOUR_DATA_SPACE_NAME');

for (ConnectApi.CdpQueryV2Row row : result.data) {
    System.debug(row.rowData);    // positional: List<Object>
}
```

- **The data space is a required argument**, not a default. Get the real name from recipe 0a.
- `row.rowData` is positional — index by SELECT order, and keep the SELECT list stable if anything downstream depends on it.
- Escape single quotes in the SQL string, and escape user input with `String.escapeSingleQuotes()` before concatenating.
- Wrap the call in `try/catch`. A malformed query throws rather than returning an empty result, and the exception message is where the actual SQL error text lives.

Path D — The CLI (fastest for one-off exploration)

Requires the community `sf data360` plugin. Handy when you are already in a terminal and want to iterate quickly, or when a result set is too big for the editor.

```
# small result sets
sf data360 query sql -o <org> --sql 'SELECT COUNT(*) FROM "ssot__AIAgentSession__dlm"'

# medium result sets – paginates properly, prefer this over OFFSET
sf data360 query sqlv2 -o <org> --sql '<your query>'

# large exports
sf data360 query async-create -o <org> --sql '<your query>'

# check a table's real columns before guessing
sf data360 query describe -o <org> --table ssot__AIAgentInteractionStep__dlm

# list the org's data spaces (also works without the plugin)
sf api request rest "/services/data/v63.0/ssot/data-spaces" -o <org>
```

describe is your tiebreaker. Several appendix conflicts — table casing, which columns actually exist on the RAG metrics DMO — are settled in ten seconds by describing the table in your own org. When a reference and this notebook disagree, the describe wins.

Section 2 — Read before you run

Seven rules that cause most bad results, and the three confidence labels every recipe carries. Two minutes here saves a lot of confused debugging later.

Seven rules

- **1 · Filter time with an ISO string literal, not date arithmetic.** `ssot__StartTimestamp__c` on `Session`, `Interaction` and `Step` is typed **string**, not timestamp. Write `>= '2026-08-12T00:00:00.000Z'`, not `> current_date - INTERVAL '7' days`. This is not a workaround: fixed-width ISO-8601 UTC strings sort chronologically, and comparing the bare column keeps whatever indexing the engine has on it. Recipes below use `'<ISO_CUTOFF>'` — compute it and paste it in.
- **2 · Never subtract two timestamps.** Same reason — they are strings. Use `datediff('millisecond', TRY_CAST(a AS TIMESTAMP), TRY_CAST(b AS TIMESTAMP))`, and remember `ssot__EndTimestamp__c` is **null on any turn that never finished** — so duration rankings silently drop the hung turn you are hunting.
- **3 · Table casing is not uniform.** `Session`, `Interaction` and `Step` use uppercase `AIAgent`. *Every other* `ssot__*AiAgent*__d` table — `Message`, `Participant`, `Moment`, `Tag`, `TagAssociation` — uses lowercase `AiAgent`. Field names always use lowercase `AiAgent`. Sources disagree on this; if a table is not found, try the other casing before assuming anything else.
- **4 · The gateway `sessionId__c` is stored quoted.** The stored value includes literal double-quotes, so match with `LIKE '%<sid>%'` or `= '"<sid>"`. A plain equals on the bare UUID finds nothing. Rows belonging to no session carry the sentinel `"no_session"` — exclude it from cost roll-ups.
- **5 · Four different values all mean "no error".** `NOT_SET`, `UNSET_VALUE` and the empty string all mean "no value" — and on top of those three, guardrail steps carry the literal string `None` in the error field, which is documented as *not* a real error. Filtering with `!= 'NOT_SET'` alone reports all four as failures that never happened.
- **6 · DurationNumber on spans is in nanoseconds.** One second is `1000000000`; divide by `1e6` for milliseconds. Forget this and every span looks a billion times too slow.
- **7 · Scope to one agent, or the numbers mean nothing.** Agent identity is **not on Session**. Every fleet-wide query in this notebook carries a `Participant` join — see `Group 0`. Without it you are averaging across every agent in the org.

The three labels

Label	What it means	How much to trust it
VERIFIED	Table names, field names <i>and</i> query shape match something that actually runs — a live describe plus an executable template or the shipped Apex service.	Run it. If it returns nothing, the data is missing, not the query.
DERIVED	Real fields, but the aggregation, ranking or duration math is ours.	Sanity-check against one session whose behaviour you already know before quoting a fleet-wide roll-up.
UNCONFIRMED	Field names are real and the shape is plausible, but no source we trust has run this against a live org — enum values, casts, and joins the references disagree about.	Treat as a starting point. Verify in your org, then pin the answer.

Why three tiers and not two. "The field names are real" and "this query works" are different claims, and collapsing them is how a confident-looking query ends up returning zero rows on a real org. A recipe can name every column correctly and still be built on an enum value your org does not use, or a function your engine spells differently. Those are UNCONFIRMED, and they say so.

Group 0 — Scope before you query

Four things to establish before anything else. Skipping these is the most common reason a correct query returns nothing — and the reason a fleet-wide number is quietly meaningless. Do this once per investigation, then reuse the values.

Zero rows almost never means "tracing is broken". In order of likelihood it means: wrong data space (0a), agent has no traffic in your window (0b, 0c), or the session is too fresh for that table to have materialized (0d and Group A). Rule those out before touching any configuration.

0a. Find the right Data Space

VERIFIED · **Answers:** which data space name to run every other query against.

Run it when: *first, on any org you have not queried before. Never assume default.*

```
-- Not SQL — a REST call. Run it from the CLI first.
sf api request rest "/services/data/v63.0/ssot/data-spaces" -o <your-org>

-- (omit --json; it is unsupported on this beta command)
```

How to read the result: Take the name value, not the label. The Query editor has a data-space picker; the Query API and the Apex service both take it as a parameter. Running against the wrong space returns zero rows for every query in this notebook and looks exactly like broken ingestion.

0b. Which agents ran, and how much

VERIFIED · **Answers:** the exact agent API names in this org, with session volume and version.

Run it when: *before any fleet-wide query — you need the API name to scope it, and the version to compare releases in Group R.*

Agent name is not on Session. The Participant DMO is where bot identity lives (ssot__AiAgentApiName__c on rows where role = AGENT). The Moment DMO carries it too, but Moment requires the Agent Optimization add-on and is often absent — Participant is the reliable source.

```

SELECT p.ssot__AiAgentApiName__c,
       p.ssot__AiAgentVersionApiName__c,
       COUNT(DISTINCT p.ssot__AiAgentSessionId__c) AS sessions
FROM ssot__AiAgentSessionParticipant__dml p
WHERE p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
      AND p.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
GROUP BY p.ssot__AiAgentApiName__c,
         p.ssot__AiAgentVersionApiName__c
ORDER BY sessions DESC

```

How to read the result: Copy the `ssot__AiAgentApiName__c` you care about — it is the `<AGENT_API_NAME>` placeholder everywhere below. Note the lowercase `AiAgent` in the table name. Two versions with traffic in the same window means you are looking at a rollout; scope to one or your averages blend them.

0c. The agent-scoping join

VERIFIED · **Answers:** the clause to bolt onto any fleet-wide recipe to restrict it to one agent.

Run it when: *every time you run a query that is not already filtered to a single session id.*

```

-- Paste after your FROM clause. `i` is the Interaction alias;
-- swap it for whatever table carries ssot__AiAgentSessionId__c.

JOIN ssot__AiAgentSessionParticipant__dml p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
  AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
  AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'

```

How to read the result: Every fleet-wide recipe in Groups C, D, E, Q, R and T below already includes this. Where a recipe queries a table with no session FK (Step, for example), reach the session through Interaction first.

0d. How fresh is this session?

VERIFIED · **Answers:** whether the session is old enough for the step-level tables to exist yet.

Run it when: *the moment someone hands you a session id — before you conclude anything from an empty result.*

```

SELECT s.ssot__Id__c,
       s.ssot__StartTimestamp__c,
       s.ssot__EndTimestamp__c,
       s.ssot__AiAgentChannelType__c,
       s.ssot__AiAgentSessionEndType__c,
       s.ssot__PreviousSessionId__c
FROM ssot__AiAgentSession__dml s
WHERE s.ssot__Id__c = '<SESSION_ID>'

```

How to read the result: If the session started within the last few hours, expect Interaction, Step, Message and Generation to be **empty** — that is normal, not a fault. Go to playbook **P0** and debug from the gateway side instead. A non-empty `ssot__PreviousSessionId__c` means this conversation is a continuation; see X5.

Group A — Setup checks: is data flowing, or is it just early?

Run these three first. But read the box below before you act on a zero — the reflex is to go and fix ingestion, and most of the time that is the wrong conclusion.

Zero rows? Check the clock before you check the config. Data Cloud DMOs do not land together. **Session, Participant and the Gateway DMOs materialize in minutes. Interaction, Step, Message, Generation and Moment take hours to days.** Platform tracing spans collect on a 5-minute cycle. One gateway child is the exception rather than merely late: GenAIGtwyRequestLLM__dml returns zero rows on every sandbox observed — the schema is provisioned but the writer is inactive.

- **Gateway rows present (A2) but no Interaction rows (B3)** → normal for a fresh session. Everything you need in the first few hours is on the gateway side: go to **P0**. Come back tomorrow for the step trace.
- **Gateway rows missing too** → now it is a real ingestion or data-space problem.
- **Spans missing, everything else present** → Agent Platform Tracing is off, or the 5-minute collection has not run yet.

A1. Do recent sessions exist?

VERIFIED · **Answers:** whether the session stream is landing at all.

Run it when: first, on any new org or after a config change.

```
SELECT ssot__Id__c,  
       ssot__StartTimestamp__c,  
       ssot__EndTimestamp__c,  
       ssot__AiAgentChannelType__c,  
       ssot__AiAgentSessionEndType__c  
FROM ssot__AIAgentSession__dml  
WHERE ssot__StartTimestamp__c >= '<ISO_CUTOFF>'  
ORDER BY ssot__StartTimestamp__c DESC  
LIMIT 20
```

How to read the result: Rows back = the session DMO is populated. Note a real `ssot__Id__c`; you will paste it into most later queries. Session lands in minutes, so a zero here — with the right data space — genuinely does point at ingestion.

Pitfall. Do not reach for `current_date - INTERVAL '7 days'` here. This column is a **string**, so comparing it to a date expression is a type mismatch. Use the ISO literal — B1, B7, C5, C6, E1 and E2 are the same. A2, A3 and the whole T and D groups are the exception: their timestamp columns really are datetimes, so date arithmetic is fine there.

A2. Is token / gateway data flowing?

VERIFIED · **Answers:** whether LLM gateway audit rows (tokens, model, prompt) are being captured.

Run it when: before any cost or token query — and first on a fresh session, because this is the stream that lands fastest.

This is the fast stream, not the fragile one. It is tempting to assume the gateway feed is the fussy one to enable. It is the opposite: gateway rows materialize in minutes and are usually the *only* thing available on a recent session. Interaction and Step are the slow ones.

```
SELECT gatewayRequestId__c,  
       sessionId__c,  
       model__c,  
       provider__c,  
       feature__c,
```

```

        totalTokens__c,
        timestamp__c
FROM GenAIGatewayRequest__dml
WHERE timestamp__c > current_date - INTERVAL '7 days'
ORDER BY timestamp__c DESC
LIMIT 20

```

How to read the result: Rows with a non-null `model__c` and `totalTokens__c` mean the gateway feed works. This table has no `ssot__` prefix, its `timestamp__c` is a genuine datetime (so date arithmetic is fine here), and its `sessionId__c` values are stored quoted.

A3. Do platform telemetry spans exist?

VERIFIED · **Answers:** whether fine-grained trace spans — the source of precise timing — are present.

Run it when: *before running any Group T timing query.*

Date arithmetic is safe here. `ssot__StartDateTime__c` on the span DMO is a real datetime — unlike the string-typed `ssot__StartTimestamp__c` on Session, Interaction and Step. A few other columns are genuine datetimes too — `ssot__CreateDate__c` on the tag DMOs, `ssot__StartTimestamp__c` on MomentInteraction, and every column on `GenAIGatewayRequest__dml`. When in doubt, check the describe: string columns need the ISO literal, datetime columns accept either.

```

SELECT ssot__Id__c,
       ssot__OperationName__c,
       ssot__ServiceName__c,
       ssot__StatusCode__c,
       ssot__DurationNumber__c,
       ssot__StartDateTime__c
FROM ssot__TelemetryTraceSpan__dml
WHERE ssot__StartDateTime__c > current_date - INTERVAL '2 days'
ORDER BY ssot__StartDateTime__c DESC
LIMIT 20

```

How to read the result: Rows back = spans are ingesting. Skim `ssot__OperationName__c` for values like `run.interaction`, `run.llmstep`, `run.action.*`. No rows means Agent Platform Tracing is not enabled (Setup → Einstein Audit, Analytics, and Monitoring), or the 5-minute collection cycle has not run.

Group B — Traceability: what happened in this session?

The bread-and-butter walk-through: session → messages → turns → steps. Everything here needs the STDM hierarchy to have materialized — if the session is only hours old, these return nothing and that is expected (Group A).

B1. Find your session (newest first)

VERIFIED · **Answers:** which session is which, with channel and how it ended.

Run it when: you have a customer complaint and a rough time, but no session id yet.

```
SELECT s.ssot__Id__c,  
       s.ssot__StartTimestamp__c,  
       s.ssot__EndTimestamp__c,  
       s.ssot__AiAgentChannelType__c,  
       s.ssot__AiAgentSessionEndType__c  
FROM ssot__AiAgentSession__dml s  
JOIN ssot__AiAgentSessionParticipant__dml p  
  ON p.ssot__AiAgentSessionId__c = s.ssot__Id__c  
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'  
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'  
WHERE s.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'  
ORDER BY s.ssot__StartTimestamp__c DESC  
LIMIT 50
```

How to read the result: Match on start time and channel to spot the session; copy its ssot__Id__c. Drop the Participant join if you want every agent's sessions — but in a multi-agent org you almost never do.

B2. All messages in a session (what was said)

VERIFIED · **Answers:** the actual customer and agent turns, in order.

Run it when: you want the human-readable transcript before diving into internals.

```
SELECT ssot__AiAgentInteractionMessageType__c,  
       ssot__ContentText__c,  
       ssot__MessageSentTimestamp__c,  
       MessageStartTimestamp__c  
FROM ssot__AiAgentInteractionMessage__dml  
WHERE ssot__AiAgentSessionId__c = '<SESSION_ID>'  
ORDER BY COALESCE(ssot__MessageSentTimestamp__c,  
                  CAST(MessageStartTimestamp__c AS VARCHAR)) ASC
```

How to read the result: Input rows are the customer, Output rows are the agent. Note the lowercase AiAgent in this table name, and that it has a direct session FK — no join needed.

Pitfall. Voice sessions order wrongly without the COALESCE. ssot__MessageSentTimestamp__c is populated on text channels only. Voice and streaming carry MessageStartTimestamp__c / MessageEndTimestamp__c — note the missing ssot__ prefix on those two. Sort on the text field alone and a voice transcript comes back in undefined order. If the CAST is rejected in your org, run two variants and pick by channel.

B3. List the turns in a session

VERIFIED · **Answers:** each planner turn (TURN rows) and the topic/subagent it was routed to.

Run it when: you want to see the routing decisions before opening any single turn.

```
SELECT ssot__Id__c,  
       ssot__TopicApiName__c,  
       ssot__StartTimestamp__c,  
       ssot__EndTimestamp__c  
FROM ssot__AIAgentInteraction__dlm  
WHERE ssot__AiAgentSessionId__c = '<SESSION_ID>'  
      AND ssot__AiAgentInteractionType__c = 'TURN'  
ORDER BY ssot__StartTimestamp__c ASC
```

How to read the result: One row per turn. `ssot__TopicApiName__c` tells you which subagent handled it — a null or `NOT_SET` here is an unrouted turn and a strong accuracy signal (see B8). Copy an `ssot__Id__c` to drill into its steps (B4). The `SESSION_END` type is filtered out.

B4. List the steps inside one turn

VERIFIED · **Answers:** the ordered internal steps (topic, LLM, action, guardrail) of a single turn.

Run it when: a specific turn misbehaved and you need its blow-by-blow.

```
SELECT ssot__AiAgentInteractionStepType__c,  
       ssot__Name__c,  
       ssot__StartTimestamp__c,  
       ssot__EndTimestamp__c,  
       ssot__GenerationId__c,  
       ssot__ErrorMessageText__c  
FROM ssot__AIAgentInteractionStep__dlm  
WHERE ssot__AiAgentInteractionId__c = '<INTERACTION_ID>'  
ORDER BY ssot__StartTimestamp__c ASC
```

How to read the result: Read top to bottom as execution order. The five live step types are `TOPIC_STEP`, `LLM_STEP`, `ACTION_STEP`, `TRUST_GUARDRAILS_STEP` and `SESSION_END`. Four-type lists circulate widely, so the fifth often gets mistaken for corruption. `ssot__GenerationId__c` is populated on `LLM_STEP` rows only; it is your key into D6.

Pitfall. Do not read `ssot__ErrorMessageText__c` as a plain error here — guardrail steps routinely carry the literal string `None`, and empty is one of `NOT_SET`, `UNSET_VALUE` or `' '`. E1 has the correct filter.

B5. Count turns and steps per session

DERIVED · **Answers:** the size and shape of a session at a glance.

Run it when: you want to know if a session was a quick two-turn chat or a 40-step marathon.

```
SELECT i.ssot__AiAgentSessionId__c,  
       COUNT(DISTINCT i.ssot__Id__c) AS turns,  
       COUNT(st.ssot__Id__c) AS steps  
FROM ssot__AIAgentInteraction__dlm i  
LEFT JOIN ssot__AIAgentInteractionStep__dlm st  
      ON st.ssot__AiAgentInteractionId__c = i.ssot__Id__c  
WHERE i.ssot__AiAgentInteractionType__c = 'TURN'  
      AND i.ssot__AiAgentSessionId__c = '<SESSION_ID>'  
GROUP BY i.ssot__AiAgentSessionId__c
```

How to read the result: A high step-to-turn ratio usually means many tool calls or planner retries — worth a closer look with B4 and D11.

Pitfall. The LEFT is load-bearing. An inner join drops any turn with zero step rows and undercounts turns — which is exactly the preview-session and not-yet-materialized case, so it would be wrong precisely when it matters.

B6. What end types does this org actually record?

VERIFIED · **Answers:** the real vocabulary of `ssot__AiAgentSessionEndType__c` in your org.

Run it when: before B7, and before you write any filter on session outcome. Always.

Why this recipe exists. Five reference sources give five different value lists for this field, and it is **absent from the live-API-verified enum table**. A hard-coded `IN ( 'Escalated', 'Abandoned' )` filter is the obvious thing to write — and on an org that records `USER_ENDED` it returns zero rows and reads as "no problem sessions today", which is the most dangerous possible answer. Discover first, filter second.

```
SELECT ssot__AiAgentSessionEndType__c, COUNT(*) AS sessions
FROM ssot__AiAgentSession__dlm
WHERE ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
GROUP BY ssot__AiAgentSessionEndType__c
ORDER BY sessions DESC
```

How to read the result: Whatever comes back is the truth for your org. Expect one of two families: `USER_ENDED` / `AGENT_ENDED`, or `Completed` / `Abandoned` / `Escalated` / `Failed`. Also expect a large null / `NOT_SET` bucket — that is usually where abandonment actually lives.

B7. Sessions that ended badly

UNCONFIRMED · **Answers:** which recent sessions ended in a state worth reviewing.

Run it when: you are triaging a day's traffic and want the problem sessions first.

```
-- Substitute the values B6 actually returned for your org.
SELECT s.ssot__Id__c,
       s.ssot__AiAgentSessionEndType__c,
       s.ssot__AiAgentChannelType__c,
       s.ssot__StartTimestamp__c
FROM ssot__AiAgentSession__dlm s
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = s.ssot__Id__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE s.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
 AND (s.ssot__AiAgentSessionEndType__c IN (<VALUES_FROM_B6>)
      OR s.ssot__AiAgentSessionEndType__c IS NULL
      OR s.ssot__AiAgentSessionEndType__c = 'NOT_SET')
ORDER BY s.ssot__StartTimestamp__c DESC
LIMIT 50
```

How to read the result: Each row is a candidate for a deep-dive — feed the `ssot__Id__c` into B2/B3. Keep the null and `NOT_SET` arms even after you know your org's values: a session that stops without an end event is the classic abandonment, and no `IN` list catches it.

Pitfall. Marked UNCONFIRMED, not VERIFIED, because the enum is unsettled. The query shape is sound; the literal values are yours to supply from B6.

B8. Routing distribution and the unrouted rate

DERIVED · **Answers:** which subagents handle traffic, and how many turns get routed nowhere.

Run it when: you suspect misrouting — this shows the pattern that B3 only shows one session at a time.

```
SELECT COALESCE(i.ssot__TopicApiName__c, '(unrouted)') AS topic,
       COUNT(*) AS turns
FROM ssot__AIAgentInteraction__dlm i
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE i.ssot__AiAgentInteractionType__c = 'TURN'
 AND i.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
GROUP BY i.ssot__TopicApiName__c
ORDER BY turns DESC
```

How to read the result: A fat (unrouted) or NOT_SET bucket is the single loudest accuracy signal you can get in one query: the planner could not decide which subagent owned the utterance. Fix by sharpening topic descriptions. A topic with near-zero traffic is either dead weight or is being starved by an overlapping neighbour.

Pitfall. Add 'NOT_SET' to the COALESCE arm if your org writes the sentinel rather than a true null — check with a quick GROUP BY first.

Group C — Time, from the step timestamps

Durations computed from Interaction and Step timestamps. These are the fallback when Agent Platform Tracing is off; when spans are available, **Group T is more accurate and needs none of this casting**. Every duration here needs an explicit cast because the timestamp columns are strings.

You cannot just subtract the timestamps. The obvious (`ssot__EndTimeStamp__c - ssot__StartTimeStamp__c`) does not work: both operands are **strings**, so it is a type error, not a precision caveat. Every recipe below casts explicitly. `TRY_CAST` returns null rather than throwing when a value will not parse, which is what you want across a whole session. Verify the cast accepts your org's timestamp format once, then trust it everywhere.

C1. Slowest turns in a session

DERIVED · **Answers:** which turns took the longest, ranked.

Run it when: a customer said 'it hung' and you need to find the offending turn.

```
SELECT ssot__Id__c,
       ssot__TopicApiName__c,
       ssot__StartTimeStamp__c,
       ssot__EndTimeStamp__c,
       datediff('millisecond',
               TRY_CAST(ssot__StartTimeStamp__c AS TIMESTAMP),
               TRY_CAST(ssot__EndTimeStamp__c AS TIMESTAMP)) AS turn_ms
FROM ssot__AIAgentInteraction__dml
WHERE ssot__AiAgentSessionId__c = '<SESSION_ID>'
      AND ssot__AiAgentInteractionType__c = 'TURN'
ORDER BY turn_ms DESC
```

How to read the result: Top row is your slowest completed turn; copy its `ssot__Id__c` into C3. Then **run C2** — the turn that never ended will not appear in this ranking at all.

Pitfall. Anything with a null `ssot__EndTimeStamp__c` sorts out of this result entirely. A turn that hung forever produces no end event, so the one turn you are hunting is the one this query cannot show you. That is what C2 is for.

C2. Turns that never ended

VERIFIED · **Answers:** turns with no end event — hung, timed out, or abandoned mid-turn.

Run it when: immediately after C1, every time. This is the blind spot in any duration ranking.

Null ends are normal and common. Sessions and turns frequently carry a null end when no end event was recorded. It does not by itself indicate a problem — it indicates you cannot compute a duration, which is why this query exists separately from C1.

```
SELECT ssot__Id__c,
       ssot__TopicApiName__c,
       ssot__StartTimeStamp__c
FROM ssot__AIAgentInteraction__dml
WHERE ssot__AiAgentSessionId__c = '<SESSION_ID>'
      AND ssot__AiAgentInteractionType__c = 'TURN'
      AND (ssot__EndTimeStamp__c IS NULL
           OR ssot__EndTimeStamp__c = 'NOT_SET')
ORDER BY ssot__StartTimeStamp__c ASC
```

How to read the result: Rows here are turns the platform never closed out. Some are benign — a session still in flight, or a user who simply left. A turn in the middle of an otherwise complete session is not benign: open its steps (B4) and look for an action that never returned.

C3. Per-step durations for one turn

DERIVED · **Answers:** which step inside a slow turn ate the time.

Run it when: C1 pointed at a turn and you want the culprit step.

```
SELECT ssot__AiAgentInteractionStepType__c,  
       ssot__Name__c,  
       ssot__StartTimestamp__c,  
       ssot__EndTimestamp__c,  
       datediff('millisecond',  
                TRY_CAST(ssot__StartTimestamp__c AS TIMESTAMP),  
                TRY_CAST(ssot__EndTimestamp__c AS TIMESTAMP)) AS step_ms  
FROM ssot__AiAgentInteractionStep__dlm  
WHERE ssot__AiAgentInteractionId__c = '<INTERACTION_ID>'  
ORDER BY step_ms DESC
```

How to read the result: The biggest step_ms is the bottleneck. An LLM_STEP on top points at model latency (confirm with T5, then check token counts in D14); an ACTION_STEP points at a slow backend action (C7 ranks those fleet-wide).

Pitfall. Step timestamps can overlap or round oddly for nested steps, and a step with no end sorts out just as in C1. Cross-check the worst offender against its span in T3 — spans carry a real numeric duration and need no arithmetic.

C4. Turn shape: how many of each step type

DERIVED · **Answers:** how many LLM, action, topic and guardrail steps a turn ran.

Run it when: you suspect a turn is doing too many tool calls or LLM round-trips.

```
SELECT ssot__AiAgentInteractionStepType__c,  
       COUNT(*) AS step_count  
FROM ssot__AiAgentInteractionStep__dlm  
WHERE ssot__AiAgentInteractionId__c = '<INTERACTION_ID>'  
GROUP BY ssot__AiAgentInteractionStepType__c  
ORDER BY step_count DESC
```

How to read the result: Many ACTION_STEP rows = a tool-heavy turn; many LLM_STEP rows = repeated planning or retries. Both add latency and tokens — confirm the retry theory against D11. You will see up to five buckets including SESSION_END.

Pitfall. A raw count. Pair it with C3 durations: one slow step usually matters more than many fast ones.

C5. Slowest turns per subagent (one agent, one day)

DERIVED · **Answers:** the worst-case latency each subagent produced.

Run it when: you are hunting a systemic slow topic, not a one-off.

```
SELECT i.ssot__TopicApiName__c,
       MAX(datediff('millisecond',
                   TRY_CAST(i.ssot__StartTimestamp__c AS TIMESTAMP),
                   TRY_CAST(i.ssot__EndTimestamp__c AS TIMESTAMP))) AS worst_ms,
       COUNT(*) AS turns
FROM ssot__AIAgentInteraction__dlm i
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE i.ssot__AiAgentInteractionType__c = 'TURN'
 AND i.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
 AND i.ssot__EndTimestamp__c IS NOT NULL
GROUP BY i.ssot__TopicApiName__c
ORDER BY worst_ms DESC
```

How to read the result: A subagent with a high `worst_ms` and many turns is a systemic latency risk — inspect its actions with C7.

Pitfall. Max is one bad sample. Read alongside C6's average and, better, T7's percentiles — a p95 problem shows in neither max nor mean.

C6. Average turn duration by subagent

DERIVED · **Answers:** typical (not just worst-case) latency per subagent.

Run it when: you want a fair comparison across topics, not skewed by one outlier.

```
SELECT i.ssot__TopicApiName__c,
       AVG(datediff('millisecond',
                   TRY_CAST(i.ssot__StartTimestamp__c AS TIMESTAMP),
                   TRY_CAST(i.ssot__EndTimestamp__c AS TIMESTAMP))) AS avg_ms,
       COUNT(*) AS turns
FROM ssot__AIAgentInteraction__dlm i
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE i.ssot__AiAgentInteractionType__c = 'TURN'
 AND i.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
 AND i.ssot__EndTimestamp__c IS NOT NULL
GROUP BY i.ssot__TopicApiName__c
ORDER BY avg_ms DESC
```

How to read the result: Sort by `avg_ms` to rank subagents by typical speed. Watch the turns count so you do not over-read a topic with two samples.

Pitfall. Averages hide the tail, and excluding null-ended turns biases the mean *downward* — the slowest turns are the likeliest to have no end event. Read C2's count alongside this.

C7. Action latency leaderboard

DERIVED · **Answers:** which specific Apex, Flow or external action is slow, ranked.

Run it when: C3 pointed at an ACTION_STEP and you want to know whether it is slow everywhere or just once.

```
SELECT st.ssot__Name__c AS action_name,
       COUNT(*) AS calls,
       AVG(datediff('millisecond',
                    TRY_CAST(st.ssot__StartTimestamp__c AS TIMESTAMP),
                    TRY_CAST(st.ssot__EndTimestamp__c AS TIMESTAMP))) AS avg_ms,
       MAX(datediff('millisecond',
                    TRY_CAST(st.ssot__StartTimestamp__c AS TIMESTAMP),
                    TRY_CAST(st.ssot__EndTimestamp__c AS TIMESTAMP))) AS worst_ms
FROM ssot__AIAgentInteractionStep__dlm st
JOIN ssot__AIAgentInteraction__dlm i
  ON i.ssot__Id__c = st.ssot__AiAgentInteractionId__c
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE st.ssot__AIAgentInteractionStepType__c = 'ACTION_STEP'
 AND st.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
 AND st.ssot__EndTimestamp__c IS NOT NULL
GROUP BY st.ssot__Name__c
ORDER BY avg_ms DESC
```

How to read the result: This is the most directly actionable latency query in the notebook: it names the action to go optimise. High avg_ms with high calls is your best return on effort. High worst_ms with a normal average points at a timeout or a cold start rather than the action itself.

Pitfall. Steps have no session FK, hence the two-hop join through Interaction. Drop both joins for a whole-org view.

C8. Time to first response

DERIVED · **Answers:** how long the user waited before seeing anything — the number they actually feel.

Run it when: a session 'felt slow' but the turn durations look fine.

```
SELECT s.ssot__Id__c,
       s.ssot__StartTimestamp__c,
       MIN(m.ssot__MessageSentTimestamp__c) AS first_agent_reply,
       datediff('millisecond',
                TRY_CAST(s.ssot__StartTimestamp__c AS TIMESTAMP),
                TRY_CAST(MIN(m.ssot__MessageSentTimestamp__c) AS TIMESTAMP))
       AS time_to_first_ms
FROM ssot__AIAgentSession__dlm s
JOIN ssot__AiAgentInteractionMessage__dlm m
  ON m.ssot__AiAgentSessionId__c = s.ssot__Id__c
WHERE s.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
 AND m.ssot__AiAgentInteractionMessageType__c = 'Output'
GROUP BY s.ssot__Id__c, s.ssot__StartTimestamp__c
ORDER BY time_to_first_ms DESC
LIMIT 50
```

How to read the result: Total turn duration is an engineering metric; this is the user-experience one. A session can have healthy per-turn numbers and still open with an eight-second silence — greeting generation, session variable resolution and the first topic decision all land before the first token.

Pitfall. Text channels only, for the same reason as B2. On voice, swap in `MessageStartTimestamp__c`.

Group T — Precise timing with telemetry spans

When you need exact, authoritative durations, use the platform spans. These carry a real numeric duration column, so none of Group C's casting applies — but remember `ssot__DurationNumber__c` is in **nanoseconds**: one second is 1000000000, and you divide by 1e6 for milliseconds. Requires Agent Platform Tracing.

T1. Slowest operation types, fleet-wide

VERIFIED · **Answers:** which kinds of operation (LLM step, action, topic) are slow across all traffic.

Run it when: you want a system-level view of where latency lives.

```
SELECT ssot__OperationName__c,
       COUNT(*)                AS spans,
       AVG(ssot__DurationNumber__c) AS avg_ns,
       MAX(ssot__DurationNumber__c) AS max_ns
FROM ssot__TelemetryTraceSpan__dlm
WHERE ssot__StartDateTime__c > current_date - INTERVAL '2 days'
GROUP BY ssot__OperationName__c
ORDER BY avg_ns DESC
```

How to read the result: Divide by 1e6 for milliseconds. Operation names like `run.llmstep` or `run.action.*` on top tell you what class of work to optimise. Other values you will see: `run.interaction`, `run.invokeActions.<name>`, `run.hybridsearch.<index>`.

Pitfall. Spans carry no agent identity, so this is genuinely org-wide and cannot be scoped the way Group C can. Use it for triage direction, then confirm per-agent with C5/C7.

T2. Get the trace id for a turn

VERIFIED · **Answers:** the internal trace id that links a turn to its spans.

Run it when: you have a slow interaction and want its full span tree.

```
SELECT ssot__Id__c,
       ssot__TelemetryTraceId__c,
       ssot__AttributeText__c
FROM ssot__AIAgentInteraction__dlm
WHERE ssot__Id__c = '<INTERACTION_ID>'
```

How to read the result: `ssot__TelemetryTraceId__c` is **usually empty** on real orgs. The real id lives inside `ssot__AttributeText__c`, which is HTML-escaped JSON — unescape it and pull the `internalTraceId` key (a hex string). Use that in T3.

Pitfall. Because the column is usually empty, any join written as `span.ssot__TelemetryTrace__c = interaction.ssot__TelemetryTraceId__c` silently returns nothing. Extract the id from the attribute text and pass it as a literal.

T3. Every span for one trace

VERIFIED · **Answers:** the complete, ordered span tree for a single trace, longest first.

Run it when: you have the trace id from T2 and want the exact breakdown.

```
SELECT ssot__OperationName__c,  
       ssot__ServiceName__c,  
       ssot__StatusCode__c,  
       ssot__DurationNumber__c,  
       ssot__Id__c,  
       ssot__TelemetryParentSpanId__c,  
       ssot__StartDateTime__c,  
       ssot__EndDateTime__c  
FROM ssot__TelemetryTraceSpan__dlm  
WHERE ssot__TelemetryTrace__c = '<TRACE_ID>'  
ORDER BY ssot__DurationNumber__c DESC
```

How to read the result: The top span is your bottleneck (nanoseconds). `ssot__TelemetryParentSpanId__c` gives you the nesting — a null parent is the root. `ssot__StatusCode__c` shows which span errored. Feed the parent/child structure into T8 to find time that belongs to no operation.

T4. Recent error spans, with the reason

VERIFIED · **Answers:** which spans failed recently, on what service, and why.

Run it when: something is throwing and you want the failing operations plus the stack.

This is the fleet-wide sweep. Use it when you do not yet know which turn failed. Once you do, **E3** is the same query scoped to a single trace — go there instead of paging through this one.

```
SELECT ssot__OperationName__c,  
       ssot__ServiceName__c,  
       ssot__StatusCode__c,  
       ssot__DurationNumber__c,  
       ssot__TelemetrySpanAttributeText__c,  
       ssot__StartDateTime__c  
FROM ssot__TelemetryTraceSpan__dlm  
WHERE ssot__StatusCode__c = 'ERROR'  
      AND ssot__StartDateTime__c > current_date - INTERVAL '2 days'  
ORDER BY ssot__StartDateTime__c DESC  
LIMIT 50
```

How to read the result: `ssot__TelemetrySpanAttributeText__c` is JSON key/value span attributes and usually carries the exception or stack. Match `ssot__OperationName__c` against E1's step names to connect the span back to the step that failed.

Pitfall. The `ssot__StatusCode__c` enum is not documented anywhere — the describe says only "execution result". 'ERROR' is the conventional OpenTelemetry value and is what E3 assumes too, but confirm it once with a `GROUP BY ssot__StatusCode__c` before reading an empty result as good news.

T5. Slowest LLM spans only

VERIFIED · **Answers:** the slowest model-inference spans specifically.

Run it when: you have narrowed the problem to LLM latency and want the worst calls.

```
SELECT ssot__OperationName__c,  
       ssot__ServiceName__c,  
       ssot__DurationNumber__c,  
       ssot__StartDateTime__c  
FROM ssot__TelemetryTraceSpan__dlm  
WHERE ssot__OperationName__c LIKE 'run.llmstep%'  
      AND ssot__StartDateTime__c > current_date - INTERVAL '2 days'  
ORDER BY ssot__DurationNumber__c DESC  
LIMIT 50
```

How to read the result: Divide by 1e6 for ms. Consistently high values point at model choice or prompt size — cross-reference with D14 (prompt-size distribution) and D5 (tokens by model) before blaming the model.

T6. Where does latency live — planner, action, or retrieval?

DERIVED · **Answers:** the latency split by the service that did the work.

Run it when: you want one query that says whether to go optimise the planner, a backend action, or the knowledge retriever.

```
SELECT ssot__ServiceName__c,  
       COUNT(*) AS spans,  
       AVG(ssot__DurationNumber__c) AS avg_ns,  
       SUM(ssot__DurationNumber__c) AS total_ns  
FROM ssot__TelemetryTraceSpan__dlm  
WHERE ssot__StartDateTime__c > current_date - INTERVAL '2 days'  
GROUP BY ssot__ServiceName__c  
ORDER BY total_ns DESC
```

How to read the result: Live service names include Atlas Reasoning Engine (planner), InvocableAction (your Apex and Flows), PromptTemplate, Einstein AI Gateway and Data Cloud (retrieval / query federation). Sort by total_ns for where the time actually goes in aggregate; by avg_ns for what is individually slow.

Pitfall. Spans nest, so total_ns double-counts: a parent run.interaction span contains its children. Use it to rank, not to sum to 100%.

T7. Latency percentiles, not averages

UNCONFIRMED · **Answers:** p50 / p95 / p99 for an operation type — the distribution, not the mean.

Run it when: before you declare latency fine because the average looks fine.

```
SELECT ssot__OperationName__c,  
       COUNT(*) AS spans,  
       PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY ssot__DurationNumber__c) AS p50_ns,  
       PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY ssot__DurationNumber__c) AS p95_ns,  
       PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY ssot__DurationNumber__c) AS p99_ns  
FROM ssot__TelemetryTraceSpan__dlm  
WHERE ssot__StartDateTime__c > current_date - INTERVAL '2 days'  
GROUP BY ssot__OperationName__c  
ORDER BY p95_ns DESC
```

How to read the result: A p95 far above p50 is the signature of a latency problem that averages hide and maxima overstate — a minority of users having a consistently bad time. That is the shape worth fixing. If p50 and p95 are close and both high, everyone is slow and it is a structural problem instead.

Pitfall. Marked DERIVED: PERCENTILE_CONT is documented as a Data 360 SQL ordered-set aggregate, but the exact WITHIN GROUP syntax should be confirmed in your org once. There is no APPROX_PERCENTILE.

T8. Unaccounted time inside a trace

DERIVED · **Answers:** time in a trace that belongs to no child span — queueing and overhead.

Run it when: the span tree does not add up: the root is slow but no child explains it.

```
-- Root span duration for the trace
SELECT ssot__Id__c, ssot__OperationName__c, ssot__DurationNumber__c AS root_ns
FROM ssot__TelemetryTraceSpan__dlm
WHERE ssot__TelemetryTrace__c = '<TRACE_ID>'
      AND (ssot__TelemetryParentSpanId__c IS NULL
           OR ssot__TelemetryParentSpanId__c = 'NOT_SET');

-- Sum of its immediate children
SELECT SUM(ssot__DurationNumber__c) AS children_ns, COUNT(*) AS children
FROM ssot__TelemetryTraceSpan__dlm
WHERE ssot__TelemetryTrace__c = '<TRACE_ID>'
      AND ssot__TelemetryParentSpanId__c = '<ROOT_SPAN_ID>';
```

How to read the result: root_ns - children_ns is time the platform spent inside the turn but outside any instrumented operation: queueing, serialization, gateway hand-offs. A large gap means the fix is not in any single action or prompt — it is in how much work the turn is asking the platform to coordinate. Reducing step count (C4) helps more than optimising any one step.

Pitfall. Only compares one level. Children have children; the arithmetic holds at each level separately, not across the whole tree at once.

Group D — Tokens & cost: what did it consume?

Cost lives on GenAIGatewayRequest__dlm — no ssot__ prefix, fields end in __c. Its timestamp__c is a real datetime, so date arithmetic is safe throughout this group. This is also the fastest-materializing data you have: on a fresh session it is all you get, and it is usually enough.

Two filters belong on every fleet-level cost query. GenAIGatewayRequest__dlm holds **every** LLM call in the org — Prompt Builder previews, sf agent test runs, eval harnesses, non-Agentforce features. Rows with no owning session carry the sentinel sessionId__c = '"no_session"' (quotes included). Without AND sessionId__c != '"no_session"' and, usually, AND feature__c = 'plannerservice', your cost numbers are inflated by traffic that has nothing to do with the agent.

D1. Every model call for a session

VERIFIED · **Answers:** each gateway request in a session with its model, template and token counts.

Run it when: you want the raw cost ledger for one conversation — or you are debugging a session too fresh for Group B.

```
SELECT gatewayRequestId__c,  
       generationGroupId__c,  
       model__c,  
       provider__c,  
       feature__c,  
       promptTemplateDevName__c,  
       promptTokens__c,  
       completionTokens__c,  
       totalTokens__c,  
       timestamp__c  
FROM GenAIGatewayRequest__dlm  
WHERE sessionId__c LIKE '%<SESSION_ID>%'  
ORDER BY timestamp__c ASC
```

How to read the result: One row per model call. generationGroupId__c ties calls to a single user turn (used in D2). promptTemplateDevName__c — values like AiCopilot__ReactTopicPrompt or Atlas__AgentGraphReasoningPrompt — is the closest you get to "which part of the agent". The LIKE is essential because the stored id is wrapped in quotes.

D2. Tokens per user turn

DERIVED · **Answers:** how many tokens each turn burned, grouped by generation group.

Run it when: you want to know which turn was the expensive one.

```
SELECT generationGroupId__c,  
       COUNT(*)           AS calls,  
       SUM(promptTokens__c) AS prompt_tokens,  
       SUM(completionTokens__c) AS completion_tokens,  
       SUM(totalTokens__c)   AS total_tokens,  
       MIN(timestamp__c)    AS turn_started  
FROM GenAIGatewayRequest__dlm  
WHERE sessionId__c LIKE '%<SESSION_ID>%'  
GROUP BY generationGroupId__c  
ORDER BY total_tokens DESC
```

How to read the result: The top generationGroupId__c is your priciest turn. A high calls count with high tokens means retries or over-decomposition — check D11. Order by turn_started instead and you get the token curve across the conversation, which is D10.

Pitfall. Confirm the totals against D1's raw rows for one turn before trusting the ranking.

D3. Total tokens for a session

DERIVED · **Answers:** the whole-conversation token bill.

Run it when: you need a single number for a cost report.

```
SELECT SUM(promptTokens__c)      AS prompt_tokens,  
       SUM(completionTokens__c) AS completion_tokens,  
       SUM(totalTokens__c)       AS total_tokens,  
       COUNT(*)                  AS calls  
FROM GenAIGatewayRequest__dlm  
WHERE sessionId__c LIKE '%<SESSION_ID>%'
```

How to read the result: One row: the session's total consumption. Compare across sessions to spot expensive conversation types. Check calls against D1's row count to confirm the LIKE matched what you expected.

D4. Most expensive requests recently

VERIFIED · **Answers:** the single biggest token calls for one agent.

Run it when: you are chasing a cost spike and want the worst individual requests.

```
SELECT gatewayRequestId__c,  
       sessionId__c,  
       model__c,  
       feature__c,  
       promptTemplateDevName__c,  
       promptTokens__c,  
       completionTokens__c,  
       totalTokens__c,  
       timestamp__c  
FROM GenAIGatewayRequest__dlm  
WHERE timestamp__c > current_date - INTERVAL '7 days'  
      AND sessionId__c != '"no_session"'  
      AND feature__c = 'plannerservice'  
ORDER BY totalTokens__c DESC  
LIMIT 50
```

How to read the result: Top rows are your heaviest calls. Strip the quotes off sessionId__c to open the conversation. If promptTokens__c dominates completionTokens__c, it is a context problem, not an output problem — D9 quantifies that.

Pitfall. Drop feature__c = 'plannerservice' if you want a whole-org cost view including Prompt Builder and prompt-template invocations. Keep != '"no_session"' either way unless you are specifically auditing non-session traffic.

D5. Tokens rolled up by model

DERIVED · **Answers:** total spend split by model and provider.

Run it when: you want to know which model is costing the most.

```
SELECT model__c,
       provider__c,
       COUNT(*) AS calls,
       SUM(totalTokens__c) AS total_tokens,
       AVG(totalTokens__c) AS avg_tokens
FROM GenAIGatewayRequest__dlm
WHERE timestamp__c > current_date - INTERVAL '7 days'
  AND sessionId__c != '"no_session"'
  AND feature__c = 'plannerservice'
GROUP BY model__c, provider__c
ORDER BY total_tokens DESC
```

How to read the result: The top model__c is your biggest consumer — a candidate for a cheaper model or prompt trimming. Read calls and avg_tokens together: few huge calls and many small ones need opposite fixes.

D6. Exact prompt and response for a step

VERIFIED · **Answers:** the literal prompt sent and the text returned, in two steps.

Run it when: an answer was wrong and you need to see exactly what the model saw and said.

On the step-to-gateway shortcut. The Step DMO does carry ssot__GenAIGatewayRequestId__c, and it does join to GenAIGatewayRequest.gatewayRequestId__c, whatever you may read elsewhere. But live data shows it is frequently NOT_SET even on LLM_STEP rows, so a join through it silently drops calls. Try it for a fast per-step lookup; when it comes back thin, fall back to the session-forward path in (b), which is authoritative.

```
-- (a) the model's RESPONSE for one step. LLM_STEP only.
SELECT st.ssot__Id__c,
       st.ssot__GenerationId__c,
       gen.responseText__c
FROM ssot__AIAgentInteractionStep__dlm st
JOIN GenAIGeneration__dlm gen
  ON gen.generationId__c = st.ssot__GenerationId__c
WHERE st.ssot__Id__c = '<STEP_ID>'
  AND st.ssot__AIAgentInteractionStepType__c = 'LLM_STEP';

-- (b) the PROMPTS for the whole session, to line up by group/time
SELECT gatewayRequestId__c, generationGroupId__c,
       promptTemplateDevName__c, prompt__c,
       model__c, totalTokens__c, timestamp__c
FROM GenAIGatewayRequest__dlm
WHERE sessionId__c LIKE '%<SESSION_ID>%'
ORDER BY timestamp__c ASC;
```

How to read the result: Query (a) gives the response through the reliable Step→Generation key. It resolves for **LLM_STEP rows only** — every other step type stores NOT_SET in ssot__GenerationId__c, so the join returns nothing and that is expected, not a fault. For the prompt, run (b) and correlate by generationGroupId__c and timestamp. If the prompt is missing context the model needed, that is your bug.

Pitfall. Both fields are HTML-escaped on the wire. prompt__c and responseText__c arrive with " and friends, and responseText__c is escaped JSON for tool-calling outputs. Unescape before reading, and do not try to parse the raw value. Prompts reach 30 KB+ on real sessions.

D7. Tokens by feature

DERIVED · **Answers:** which Agentforce feature or capability consumes the most tokens.

Run it when: you want to attribute cost to a capability rather than a model.

```
SELECT feature__c,  
       COUNT(*)           AS calls,  
       SUM(totalTokens__c) AS total_tokens,  
       AVG(totalTokens__c) AS avg_tokens  
FROM GenAIGatewayRequest__dlm  
WHERE timestamp__c > current_date - INTERVAL '7 days'  
      AND sessionId__c != '"no_session"'  
GROUP BY feature__c  
ORDER BY total_tokens DESC
```

How to read the result: High avg_tokens means each call is heavy; high calls means it is simply used a lot. plannerservice is the agent's own reasoning; PromptTemplateGenerationsInvocable and CopilotForDigitalChannels are the usual companions. (Guardrails and Citations is documented on the sibling GenAIGeneration.feature__c, not here.)

Pitfall. No feature__c filter here on purpose — the point is to see the split. That means this one *does* include non-planner traffic within your sessions.

D8. Tokens by prompt template

DERIVED · **Answers:** which prompt template consumes the most tokens — the best available proxy for 'which part of the agent'.

Run it when: you want to attribute cost below the model level.

```
SELECT promptTemplateDevName__c,  
       COUNT(*)           AS calls,  
       SUM(totalTokens__c) AS total_tokens,  
       AVG(totalTokens__c) AS avg_tokens  
FROM GenAIGatewayRequest__dlm  
WHERE timestamp__c > current_date - INTERVAL '7 days'  
      AND sessionId__c != '"no_session"'  
      AND feature__c = 'plannerservice'  
GROUP BY promptTemplateDevName__c  
ORDER BY total_tokens DESC
```

How to read the result: Gateway rows carry the prompt template, not the subagent name. A template with high avg_tokens is a prompt-trimming target; pair with D9 to see whether the weight is in the input or the output.

Pitfall. Per-subagent attribution is still unreliable. There is no direct GatewayRequest→Interaction edge; the documented two-hop path runs through Step.sso__GenAiGatewayRequestId__c, which is frequently NOT_SET. If you build it, measure what fraction of gateway rows you actually matched before quoting the numbers.

D9. Prompt vs completion ratio by template

DERIVED · **Answers:** whether a template's cost is input bloat or verbose output.

Run it when: D8 flagged an expensive template and you need to know which fix applies.

```
SELECT promptTemplateDevName__c,  
       COUNT(*)                AS calls,  
       AVG(promptTokens__c)    AS avg_prompt,  
       AVG(completionTokens__c) AS avg_completion,  
       AVG(promptTokens__c) / NULLIF(AVG(completionTokens__c), 0) AS ratio  
FROM GenAIGatewayRequest__dlm  
WHERE timestamp__c > current_date - INTERVAL '7 days'  
  AND sessionId__c != 'no_session'  
  AND feature__c = 'plannerservice'  
GROUP BY promptTemplateDevName__c  
ORDER BY avg_prompt DESC
```

How to read the result: This is the query that tells you what to actually change. A high ratio (input dwarfing output) means context bloat — trim instructions, tool definitions, or conversation history. A low ratio with high totals means the model is writing too much — constrain the output format. Totals alone, as in D8, cannot distinguish the two, and the fixes are opposite.

D10. Token growth across a conversation

DERIVED · **Answers:** whether prompt size is climbing turn over turn inside one session.

Run it when: a long conversation got expensive and you suspect history accumulation.

```
SELECT generationGroupId__c,  
       MIN(timestamp__c) AS turn_started,  
       SUM(promptTokens__c) AS prompt_tokens,  
       SUM(totalTokens__c) AS total_tokens,  
       COUNT(*)           AS calls  
FROM GenAIGatewayRequest__dlm  
WHERE sessionId__c LIKE '%<SESSION_ID>%'  
  AND feature__c = 'plannerservice'  
GROUP BY generationGroupId__c  
ORDER BY turn_started ASC
```

How to read the result: Read the prompt_tokens column top to bottom. A gentle rise is normal — history grows. A steep, linear climb means the full transcript is being replayed into every prompt with no trimming, and the tenth turn costs many times the first. That is the most common runaway-cost pattern there is, and it compounds with session length. Fix it in the agent's context configuration, not in the prompt text.

Pitfall. Ordering by MIN(timestamp__c) assumes generation groups do not interleave. They rarely do within one session, but check the raw D1 ledger if the curve looks non-monotonic.

D11. Retries and duplicate calls

DERIVED · **Answers:** turns where the planner made several near-identical model calls.

Run it when: C4 showed many LLM_STEP rows in one turn, or D2 showed high calls with high tokens.

```
SELECT generationGroupId__c,  
       COUNT(*)                AS calls,  
       COUNT(DISTINCT model__c) AS models,  
       MIN(LENGTH(prompt__c))  AS min_prompt_chars,  
       MAX(LENGTH(prompt__c))  AS max_prompt_chars,  
       MIN(timestamp__c)       AS first_call,  
       MAX(timestamp__c)       AS last_call,  
       SUM(totalTokens__c)     AS total_tokens  
FROM GenAIGatewayRequest__dlm  
WHERE sessionId__c LIKE '%<SESSION_ID>%'  
   AND feature__c = 'plannerservice'  
GROUP BY generationGroupId__c  
HAVING COUNT(*) > 2  
ORDER BY calls DESC
```

How to read the result: Several calls in one group with **near-identical prompt lengths**, seconds apart, is a retry — the planner asked the same question again, and you paid twice. Steadily *growing* prompt lengths within a group is normal ReAct decomposition: each step adds its observation. The distinction matters because retries are a bug and decomposition is design.

Pitfall. Marked DERIVED and slightly heuristic — LENGTH() on a 30 KB prompt is a coarse fingerprint. Confirm by reading the actual prompts from D6(b) for one suspicious group.

D12. Cost per resolved conversation

DERIVED · **Answers:** tokens divided by conversations that actually ended well.

Run it when: you are reporting on cost and want a number that improves when the agent gets better.

```
-- Step 1: total planner tokens in the window (one agent)  
SELECT SUM(g.totalTokens__c) AS total_tokens, COUNT(*) AS calls  
FROM GenAIGatewayRequest__dlm g  
WHERE g.timestamp__c > current_date - INTERVAL '7 days'  
   AND g.sessionId__c != '"no_session"'  
   AND g.feature__c = 'plannerservice';  
  
-- Step 2: sessions that ended well (use YOUR values from B6)  
SELECT COUNT(*) AS resolved_sessions  
FROM ssot__AiAgentSession__dlm s  
JOIN ssot__AiAgentSessionParticipant__dlm p  
  ON p.ssot__AiAgentSessionId__c = s.ssot__Id__c  
  AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'  
  AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'  
WHERE s.ssot__StartTimeStamp__c >= '<ISO_CUTOFF>'  
   AND s.ssot__AiAgentSessionEndType__c IN (<RESOLVED_VALUES_FROM_B6>);
```

How to read the result: Divide step 1 by step 2. Total tokens is a vanity metric — it goes up when the agent is popular and down when it is broken. Tokens per *resolved* conversation goes down only when the agent genuinely gets more efficient, which is the number worth putting in front of anyone.

Pitfall. Two queries because the gateway rows cannot be agent-scoped directly. In a single-agent org the numerator is clean; in a multi-agent org you will need to restrict step 1 by botVersionId__c (see R1) or accept that it is an org-level numerator.

D13. Sampling-parameter audit

VERIFIED · **Answers:** temperature, generation count and penalties actually in use per template.

Run it when: *output is inconsistent run to run, or cost is higher than the token counts explain.*

```
SELECT promptTemplateDevName__c,
       model__c,
       temperature__c,
       numGenerations__c,
       frequencyPenalty__c,
       presencePenalty__c,
       COUNT(*) AS calls
FROM GenAIGatewayRequest__dlm
WHERE timestamp__c > current_date - INTERVAL '7 days'
      AND sessionId__c != 'no_session'
GROUP BY promptTemplateDevName__c, model__c, temperature__c,
         numGenerations__c, frequencyPenalty__c, presencePenalty__c
ORDER BY calls DESC
```

How to read the result: numGenerations__c above 1 multiplies your cost invisibly — the model produces N candidates and you are billed for all of them. A stray high temperature__c explains "why does it answer differently every time" and makes your evals flaky. Several distinct parameter rows for one template means something is overriding config inconsistently.

D14. Prompt size distribution

UNCONFIRMED · **Answers:** the p50 / p95 / max prompt size per template, in characters.

Run it when: *T5 showed slow LLM spans and you want to know if prompt size is the cause.*

```
SELECT promptTemplateDevName__c,
       COUNT(*) AS calls,
       PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY LENGTH(prompt__c)) AS p50_chars,
       PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY LENGTH(prompt__c)) AS p95_chars,
       MAX(LENGTH(prompt__c)) AS max_chars
FROM GenAIGatewayRequest__dlm
WHERE timestamp__c > current_date - INTERVAL '7 days'
      AND sessionId__c != 'no_session'
      AND feature__c = 'plannerservice'
GROUP BY promptTemplateDevName__c
ORDER BY p95_chars DESC
```

How to read the result: Model latency scales with input length, so a template with a fat p95 is both your cost and your latency problem in one place. A p95 far above p50 means most calls are fine and a minority carry an enormous context — usually long conversations (D10) or a tool definition list that grew.

Pitfall. Same PERCENTILE_CONT caveat as T7. If the syntax is rejected, fall back to AVG and MAX, which are safe everywhere.

Group E — Errors & adherence: what actually went wrong?

Two failure sources: step-level errors and the guardrail adherence signal. Both are easy to get wrong in the same direction — reporting problems that are not problems. The filters below are the careful versions.

"Not NOT_SET" is not the same as "an error". Three values mean empty — NOT_SET, UNSET_VALUE and the empty string — and TRUST_GUARDRAILS_STEP rows routinely carry the literal Python string None in the error field, which is explicitly documented as not a real error. A bare != 'NOT_SET' filter returns all of them and calls them failures. Real action failures live on ACTION_STEP rows.

E1. Every genuinely errored step

VERIFIED · **Answers:** which steps actually failed across recent traffic, for one agent.

Run it when: you want a fleet-wide error list to triage.

```
SELECT st.ssot__AiAgentInteractionId__c,
       st.ssot__AiAgentInteractionStepType__c,
       st.ssot__Name__c,
       st.ssot__ErrorMessageText__c,
       st.ssot__StartTimestamp__c
FROM ssot__AiAgentInteractionStep__dlm st
JOIN ssot__AiAgentInteraction__dlm i
  ON i.ssot__Id__c = st.ssot__AiAgentInteractionId__c
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE st.ssot__AiAgentInteractionStepType__c = 'ACTION_STEP'
 AND st.ssot__ErrorMessageText__c NOT IN ('NOT_SET', 'None', 'UNSET_VALUE', '')
 AND st.ssot__ErrorMessageText__c IS NOT NULL
 AND st.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
ORDER BY st.ssot__StartTimestamp__c DESC
LIMIT 100
```

How to read the result: Now every row is a genuine action failure. Read ssot__ErrorMessageText__c for the message, ssot__Name__c for which action, and use ssot__AiAgentInteractionId__c to open the surrounding turn (B4). Then find the matching span in T4 for the stack.

Pitfall. To hunt LLM-step failures instead, swap the step-type filter for != 'TRUST_GUARDRAILS_STEP' — but expect noise: outside ACTION_STEP the error field is not a reliable failure signal.

E2. Low instruction-adherence turns

DERIVED · **Answers:** turns where the trust-layer guardrail flagged the model as ignoring its instructions.

Run it when: the agent went off-script and you want to catch it systematically.

The same applies to LLM_STEP input and output. ssot__InputValueText__c and ssot__OutputValueText__c on LLM_STEP rows are raw Python dict strings too (the internal engine representation). Only ACTION_STEP input/output is real JSON — see X1.

```
-- Candidate finder. Confirm each hit before acting (see below).
SELECT st.ssot__AiAgentInteractionId__c,
       st.ssot__Name__c,
       st.ssot__OutputValueText__c,
```

```

        st.ssot__StartTimestamp__c
FROM ssot__AIAgentInteractionStep__dlm st
WHERE st.ssot__AIAgentInteractionStepType__c = 'TRUST_GUARDRAILS_STEP'
      AND st.ssot__OutputValueText__c LIKE '%LOW%'
      AND st.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
ORDER BY st.ssot__StartTimestamp__c DESC
LIMIT 50

```

How to read the result: A LOW in the guardrail output means the model strayed from its instructions on that turn — open the interaction (B4) and the prompt/response (D6) to see how. Values are HIGH and LOW — a MEDIUM is likely by analogy with the trust-layer categories but is not attested — and the step is normally named InstructionAdherence.

Pitfall. This field is a Python dict string, not JSON, and it is HTML-escaped. The real shape is {'name': 'InstructionAdherence', 'value': 'HIGH', 'explanation': '...'}, documented verbatim. So: (1) do not `JSON.parse()` it; (2) the coarse `LIKE '%LOW%'` above also matches the word LOW appearing in the free-text explanation, so unescape each hit and check that the 'value' key — not the prose — is 'LOW'. The sibling step fields are HTML-escaped, so this one probably is too — that part is inference, not documentation. Look at one row in your org, then tighten the pattern once and stop guessing.

E3. Connect a failing step to its span

VERIFIED · **Answers:** the root-cause detail behind a failure — service, exception, stack.

Run it when: E1 gave you a failing step and you want to know why it failed, not just that it did.

Scoped to one trace on purpose. The fleet-wide version of this sweep is T4. Use that one to discover that something is failing; use this one once you already know which turn failed and want its stack.

```

-- 1. Get the trace id for the failing turn (see T2 — usually
--    inside ssot__AttributeText__c, not the TraceId column).
-- 2. Then pull the error spans for that trace:

SELECT ssot__OperationName__c,
       ssot__ServiceName__c,
       ssot__StatusCode__c,
       ssot__TelemetrySpanAttributeText__c,
       ssot__DurationNumber__c,
       ssot__StartDateTime__c
FROM ssot__TelemetryTraceSpan__dlm
WHERE ssot__TelemetryTrace__c = '<TRACE_ID>'
      AND ssot__StatusCode__c = 'ERROR'
ORDER BY ssot__StartDateTime__c ASC

```

How to read the result: `ssot__TelemetrySpanAttributeText__c` usually names the failing service and carries the exception. Match `ssot__OperationName__c` (`run.action.<name>`) against E1's `ssot__Name__c` to line the span up with the step.

Group Q — Quality: was the answer any good?

Four independent quality signals, strongest evidence last: LLM-assigned quality tags, trust-layer detectors, RAG grounding scores, and actual human feedback. The tag and moment recipes need the Agent Optimization add-on; the trust-layer and feedback recipes do not.

Q1. Session-level quality scores with reasons

VERIFIED · **Answers:** the 1–5 quality tags attached to a session, and the LLM's rationale.

Run it when: you want a graded view of where a conversation was weak.

```
SELECT ta.ssot__AiAgentSessionId__c,  
       t.ssot__Value__c AS quality_score,  
       ta.ssot__IsPassed__c,  
       ta.ssot__AssociationReasonText__c  
FROM ssot__AiAgentTagAssociation__dml ta  
JOIN ssot__AiAgentTag__dml t  
  ON t.ssot__Id__c = ta.ssot__AiAgentTagId__c  
WHERE ta.ssot__AiAgentSessionId__c = '<SESSION_ID>'  
ORDER BY t.ssot__Value__c ASC
```

How to read the result: Low scores first. Score labels: **5 = High, 3–4 = Medium, 2 = Low, 1 = Very Low.**

ssot__AssociationReasonText__c is the LLM's written explanation for the grade and is often more useful than the number. ssot__IsPassed__c is a ready-made pass/fail for evaluation-style tags.

Pitfall. What the session filter actually returns is contested — measure it. One reference describes TagAssociation as polymorphic, with exactly one of the Session, Moment or Interaction FK populated per row; if that holds, this query can never see moment-scoped tags. Two other sources disagree — one calls the session FK "denormalized for efficient filtering", and the shipped Apex service builds its per-moment quality distribution using exactly this session filter. Run this, run Q2, compare the counts: one minute settles it for your org. Either way, do not read a session-keyed query as per-moment detail — that is what Q2 is for. Note also that tag definitions cover more than quality — intent category tags share this table — so join through TagDefinition if you want only the score tags.

Q2. Per-moment quality, and turns-per-intent efficiency

DERIVED · **Answers:** how many turns the agent needed per user intent, with each moment's score.

Run it when: the conversation ended fine but felt laborious — this quantifies that.

```
-- Step 1: moments for the session (Agent Optimization add-on)
SELECT m.ssot__Id__c AS moment_id,
       m.ssot__RequestSummaryText__c,
       m.ssot__ResponseSummaryText__c,
       m.ssot__AiAgentApiName__c
FROM ssot__AiAgentMoment__dml m
WHERE m.ssot__AiAgentSessionId__c = '<SESSION_ID>';

-- Step 2: turns per moment + the moment's quality tag
SELECT mi.ssot__AiAgentMomentId__c,
       COUNT(DISTINCT mi.ssot__AiAgentInteractionId__c) AS turns_in_moment,
       MAX(t.ssot__Value__c) AS quality_score
FROM ssot__AiAgentMomentInteraction__dml mi
LEFT JOIN ssot__AiAgentTagAssociation__dml ta
  ON ta.ssot__AiAgentMomentId__c = mi.ssot__AiAgentMomentId__c
LEFT JOIN ssot__AiAgentTag__dml t
  ON t.ssot__Id__c = ta.ssot__AiAgentTagId__c
WHERE mi.ssot__AiAgentMomentId__c IN (<MOMENT_IDS_FROM_STEP_1>)
GROUP BY mi.ssot__AiAgentMomentId__c
ORDER BY turns_in_moment DESC
```

How to read the result: A moment is one user intent; a turn is one exchange. `turns_in_moment` well above 1 means the agent needed several rounds to satisfy a single intent — clarifying questions, failed action attempts, or a topic that keeps handing back. High turns *and* a low quality score is your worst combination and the best place to spend improvement effort. `ssot__RequestSummaryText__c` gives you the intent in plain language without reading the transcript.

Pitfall. Moment DMOs require the Agent Optimization add-on and materialize in hours to days — expect zero rows on a fresh session or an org without the add-on. Both are normal.

Q3. Lowest-scoring sessions

DERIVED · **Answers:** which sessions got the worst quality grades overall.

Run it when: you want to prioritise the worst conversations for review.

```
SELECT ta.ssot__AiAgentSessionId__c,
       AVG(TRY_CAST(t.ssot__Value__c AS DOUBLE)) AS avg_score,
       COUNT(*) AS graded_tags
FROM ssot__AiAgentTagAssociation__dml ta
JOIN ssot__AiAgentTag__dml t
  ON t.ssot__Id__c = ta.ssot__AiAgentTagId__c
WHERE ta.ssot__AiAgentSessionId__c IS NOT NULL
  AND ta.ssot__CreatedDate__c >= '<ISO_CUTOFF>'
GROUP BY ta.ssot__AiAgentSessionId__c
ORDER BY avg_score ASC
LIMIT 25
```

How to read the result: The lowest `avg_score` sessions are your review queue — feed each id into B2 and Q1. An org-wide average below 4.0 signals a systemic quality problem rather than a few bad conversations.

Pitfall. The cast is required. `ssot__value__c` is a *string* column holding "1"-"5", so a bare `AVG( )` fails. The shipped Apex service deliberately avoids this — it groups by value, counts, and averages client-side. If `TRY_CAST` misbehaves, do the same: `GROUP BY t.sso__value__c, COUNT( * )` and weight outside SQL. Non-numeric tag values (intent categories) cast to null and drop out, which is the behaviour you want here.

Q4. Trust-layer detectors: toxicity, PII, prompt defense, task resolution

VERIFIED · **Answers:** the safety and resolution scores the trust layer assigned to each generation.

Run it when: you want an accuracy signal that does not depend on the Agent Optimization add-on.

```
-- Generation ids come from the session's LLM_STEP rows (see D6).
SELECT cc.parent__c,
       cc.detectorType__c,
       cc.category__c,
       cc.value__c,
       cc.timestamp__c
FROM GenAIContentCategory__dlm cc
WHERE cc.parent__c IN (<GENERATION_IDS>)
ORDER BY cc.detectorType__c, cc.value__c DESC
```

How to read the result: `detectorType__c` takes five live values: TOXICITY, PII, PROMPT_DEFENSE, InstructionAdherence and TaskResolution. TaskResolution is the closest thing the platform gives you to "did the agent actually solve it" — it is the highest-value signal in this group, and the one most often overlooked. `value__c` is a string "0.0"-"1.0"; cast it. `category__c` carries the sub-label (violence, safety_score, or Low/Medium/High).

Pitfall. `parent__c` is polymorphic: it points at a `generationId__c` for non-toxicity detectors, and at a `GenAIContentQuality.id__c` for toxicity sub-categories. Query both parents if you want the full picture — see Q5 for the quality rows.

Q5. Was toxicity detected on any generation?

VERIFIED · **Answers:** the per-generation content-quality row, input and output side.

Run it when: a compliance review, or you need the parent ids for Q4's toxicity sub-categories.

```
SELECT cq.id__c,
       cq.parent__c AS generation_id,
       cq.contentType__c,
       cq.isToxicityDetected__c,
       cq.feature__c,
       cq.timestamp__c
FROM GenAIContentQuality__dlm cq
WHERE cq.parent__c IN (<GENERATION_IDS>)
ORDER BY cq.timestamp__c ASC
```

How to read the result: `contentType__c` is INPUT or OUTPUT; `isToxicityDetected__c` is populated on OUTPUT rows only and is the string "true"/"false", not a boolean. Take `id__c` from any row of interest and use it as a `parent__c` in Q4 to get the toxicity breakdown by category.

Q6. What did users actually say about the answers?

VERIFIED · **Answers:** thumbs up/down and free-text feedback attached to generations.

Run it when: you want ground truth rather than a model's opinion of a model.

```
SELECT f.feedbackId__c,  
       f.generationId__c,  
       f.feedback__c,  
       f.action__c,  
       f.source__c,  
       fd.feedbackText__c,  
       fd.appFeedback__c,  
       f.timestamp__c  
FROM GenAIFeedback__dml f  
LEFT JOIN GenAIFeedbackDetail__dml fd  
      ON fd.parent__c = f.feedbackId__c  
WHERE f.generationId__c IN (<GENERATION_IDS>)  
ORDER BY f.timestamp__c DESC
```

How to read the result: feedback__c is UP / DOWN; action__c carries richer signals like REGENERATE or COPY — a regenerate is a silent thumbs-down, and a copy is a silent thumbs-up. feedbackText__c is the user's own words and is worth more than any automated score in this group. Sparse by nature: most users never rate anything, so treat volume as a floor on dissatisfaction, not a measure of it.

Pitfall. Generation ids come through the Step→Generation path (D6), so this inherits the same LLM_STEP-only and materialization-lag constraints.

Q7. RAG grounding: faithfulness, relevancy, precision

DERIVED · **Answers:** how well retrieved knowledge grounded the answers.

Run it when: you suspect the knowledge base, not the model, is the problem.

DLO and DMO are different query surfaces. The observability queries in the Apex service run against Data Lake objects (__dll, camelCase fields like FaithfulnessRelevancyScoreNumber__c with no ssot__ prefix). Everything in this notebook queries DMOs (__dml, ssot____c). Do not copy field names between the two.

```
SELECT q.ssot__RetrieverApiName__c,  
       q.ssot__FaithfulnessRelevancyScoreNumber__c AS faithfulness,  
       q.ssot__AnswerRelevancyScoreNumber__c      AS answer_relevancy,  
       q.ssot__ContextPrecisionScoreNumber__c      AS context_precision,  
       q.ssot__UserUtteranceText__c,  
       q.ssot__AgentGeneratedResponseText__c  
FROM ssot__AiRetrieverQualityMetric__dml q  
WHERE q.ssot__FaithfulnessRelevancyScoreNumber__c < 0.8  
ORDER BY q.ssot__FaithfulnessRelevancyScoreNumber__c ASC  
LIMIT 50
```

How to read the result: Scores run 0–1 and these are real numeric columns, so they aggregate cleanly. The published thresholds: **faithfulness < 0.8 = hallucination risk** (the answer was not grounded in what was retrieved); **answer relevancy < 0.7 = knowledge gap**. Low context precision means the retriever pulled irrelevant chunks. ssot__UserUtteranceText__c shows the question that triggered it.

Pitfall. The columns are solid; the scoping is the hard part. This DMO's join key is `ssot__AiGatewayRequestId__c` — but reaching a session through it means going via `Step.ssot__GenAiGatewayRequestId__c`, which is frequently `NOT_SET`, so expect to lose rows. The `RetrieverTraceId__c` key you may see elsewhere is not a competing answer — it belongs to the Data Lake surface, not this one (see the note below). Until you have measured your join coverage, let the score threshold be the scope: the worst-grounded answers are worth reading whichever agent produced them.

Group X — Deep inspection: the fields nobody looks at

When the trace looks healthy but the answer was still wrong, the cause is almost always in one of these six places — and they are the fields almost nobody queries. Between them they cover most real root causes.

X1. What went into and out of an action

VERIFIED · **Answers:** the structured JSON input and output of every ACTION_STEP in a turn.

Run it when: the action ran without error but the answer was still wrong.

```
SELECT ssot__Name__c,  
       ssot__InputValueText__c,  
       ssot__OutputValueText__c,  
       ssot__ErrorMessageText__c,  
       ssot__StartTimestamp__c  
FROM ssot__AIAgentInteractionStep__dml  
WHERE ssot__AiAgentInteractionId__c = '<INTERACTION_ID>'  
      AND ssot__AiAgentInteractionStepType__c = 'ACTION_STEP'  
ORDER BY ssot__StartTimestamp__c ASC
```

How to read the result: This is where most wrong-answer investigations should start and rarely do. **On ACTION_STEP rows these two fields are genuine structured JSON** (unlike LLM_STEP, where they are Python dict strings). Read the input: did the planner pass the member id you expected, the right date format, the right plan code? A successful action called with wrong arguments produces a confidently wrong answer and no error anywhere in the trace.

Pitfall. HTML-escaped on the wire — unescape before parsing. And note this is the correct destination for a wrong-answer investigation; D6 only resolves for LLM_STEP rows.

X2. What the planner tried to call

VERIFIED · **Answers:** the tool-call payloads the planner constructed, from the gateway side.

Run it when: you need X1's answer but the session is too fresh for Step rows to exist.

```
SELECT md.parent__c AS gateway_request_id,  
       md.metadataType__c,  
       md.metadata__c,  
       md.feature__c,  
       md.timestamp__c  
FROM GenAIGtwyRequestMetadata__dml md  
WHERE md.parent__c IN (  
      SELECT gatewayRequestId__c  
      FROM GenAIGatewayRequest__dml  
      WHERE sessionId__c LIKE '%<SESSION_ID>%')  
      AND md.metadataType__c = 'ToolCall'  
ORDER BY md.timestamp__c ASC
```

How to read the result: metadata__c is the JSON tool-call payload — the function the planner selected and the arguments it built. This is the gateway-side twin of X1 and it **materializes in minutes**, which makes it the only way to answer "what did it try to do" on a session from an hour ago. Compare the arguments against what the user actually said (B2, or prompt__c in D6b).

Pitfall. If the subquery form is rejected, run the inner query first and paste the ids as a literal IN list. metadataType__c has other values — drop the filter to see what else your org emits.

X3. Variable state through the turn

VERIFIED · **Answers:** the session variables and the before/after variable snapshot at each step.

Run it when: context seems to be getting lost between turns, or a variable was never populated.

```
-- Session-level bootstrap variables
SELECT ssot__Id__c, ssot__VariableText__c
FROM ssot__AIAgentSession__dlm
WHERE ssot__Id__c = '<SESSION_ID>';

-- Per-step variable snapshots
SELECT ssot__AiAgentInteractionStepType__c,
       ssot__Name__c,
       ssot__PreStepVariableText__c,
       ssot__PostStepVariableText__c,
       ssot__StartTimestamp__c
FROM ssot__AIAgentInteractionStep__dlm
WHERE ssot__AiAgentInteractionId__c = '<INTERACTION_ID>'
ORDER BY ssot__StartTimestamp__c ASC;
```

How to read the result: Diff `ssot__PreStepVariableText__c` against `ssot__PostStepVariableText__c` down the step sequence and you can watch state appear, change, and disappear. The step where an expected variable is still empty is the step that failed to set it — which is usually several steps before the one that produced the wrong answer. Session `ssot__VariableText__c` holds the channel bootstrap dict (resolved locale, DST offset, display capabilities).

Pitfall. JSON, HTML-escaped. Also note that Builder-preview-only keys such as `__supports_result_display__` appearing here mean you are looking at a Builder session, not real channel traffic — worth knowing before you generalise from it.

X4. Did the user have to ask twice?

DERIVED · **Answers:** repeated or near-repeated user utterances within one session.

Run it when: you want a cheap, high-signal proxy for 'the agent did not actually answer'.

```
SELECT ssot__ContentText__c,
       COUNT(*) AS times_said,
       MIN(ssot__MessageSentTimestamp__c) AS first_said,
       MAX(ssot__MessageSentTimestamp__c) AS last_said
FROM ssot__AiAgentInteractionMessage__dlm
WHERE ssot__AiAgentSessionId__c = '<SESSION_ID>'
      AND ssot__AiAgentInteractionMessageType__c = 'Input'
GROUP BY ssot__ContentText__c
HAVING COUNT(*) > 1
ORDER BY times_said DESC
```

How to read the result: A user repeating themselves is the clearest unprompted signal that the previous answer missed. It needs no add-on, no LLM judge and no human rating — the user told you. Rephrasings will not group exactly, so treat this as a floor: the real repeat rate is higher than what exact matching finds.

Pitfall. Exact string grouping only. "yes" and "ok" will show up as false positives in most sessions — filter short strings out, or eyeball the results.

X5. Did this conversation start somewhere else?

VERIFIED · **Answers:** the chain of prior sessions for the same conversation.

Run it when: you are measuring repeat contact, or a session looks suspiciously short.

```
SELECT ssot__Id__c,  
       ssot__PreviousSessionId__c,  
       ssot__StartTimestamp__c,  
       ssot__EndTimestamp__c,  
       ssot__AiAgentSessionEndType__c,  
       ssot__IndividualId__c  
FROM ssot__AIAgentSession__dml  
WHERE ssot__Id__c = '<SESSION_ID>'  
      OR ssot__PreviousSessionId__c = '<SESSION_ID>'  
ORDER BY ssot__StartTimestamp__c ASC
```

How to read the result: Follow `ssot__PreviousSessionId__c` backwards to reconstruct the full conversation. This matters for measurement: a user who abandons and reconnects looks like two short, healthy sessions when it is really one failure. Any deflection or abandonment rate computed per-session without walking this chain is optimistic.

X6. PII and safety posture

VERIFIED · **Answers:** whether masking and safety scoring were actually on for these calls.

Run it when: a compliance review — especially on an agent handling health or financial data.

```
SELECT promptTemplateDevName__c,  
       enablePiiMasking__c,  
       enableInputSafetyScoring__c,  
       enableOutputSafetyScoring__c,  
       COUNT(*) AS calls  
FROM GenAIGatewayRequest__dml  
WHERE timestamp__c > current_date - INTERVAL '30 days'  
      AND sessionId__c != '"no_session"'  
GROUP BY promptTemplateDevName__c, enablePiiMasking__c,  
         enableInputSafetyScoring__c, enableOutputSafetyScoring__c  
ORDER BY calls DESC
```

How to read the result: All three are strings "true"/"false", not booleans. Any template showing `false` is sending unmasked or unscored content to the model. When masking is on, `maskedPrompt__c` is populated and is the field to read in D6 instead of `prompt__c`; when it is off, that column is empty and the raw prompt is all there is — which is also what gets stored in Data Cloud.

Group R — Baseline & regression: did my change help?

Everything above is diagnostic — it tells you what is wrong now. This group is the other half of the job: proving a change worked. All four compare two agent versions or two time windows.

R1. Tokens per turn, by agent version

DERIVED · **Answers:** whether the new version got cheaper or more expensive.

Run it when: you shipped a new agent version and want the cost verdict.

```
SELECT botVersionId__c,
       COUNT(DISTINCT generationGroupId__c) AS turns,
       COUNT(*) AS calls,
       SUM(totalTokens__c) AS total_tokens,
       SUM(totalTokens__c) * 1.0
       / NULLIF(COUNT(DISTINCT generationGroupId__c), 0) AS tokens_per_turn,
       AVG(promptTokens__c) AS avg_prompt_tokens
FROM GenAIGatewayRequest__dlm
WHERE timestamp__c > current_date - INTERVAL '14 days'
      AND sessionId__c != 'no_session'
      AND feature__c = 'plannerservice'
GROUP BY botVersionId__c
ORDER BY MIN(timestamp__c) DESC
```

How to read the result: tokens_per_turn is the comparison that survives changes in traffic volume — raw totals move with adoption and tell you nothing about efficiency. Watch calls per turn too: if the new version uses more model calls to do the same work, the planner is decomposing more than it needs to. Map version ids to names with 0b.

Pitfall. Choose a window covering both versions with meaningful traffic in each. During a phased rollout the versions are running side by side, which is ideal — same period, same traffic mix, honest comparison.

R2. Latency percentiles, week over week

UNCONFIRMED · **Answers:** whether p95 latency moved after a change.

Run it when: confirming a performance fix, or catching a regression before users report it.

```
-- Run once per window and compare. Change the two dates only.
SELECT ssot__OperationName__c,
       COUNT(*) AS spans,
       PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY ssot__DurationNumber__c) AS p50_ns,
       PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY ssot__DurationNumber__c) AS p95_ns
FROM ssot__TelemetryTraceSpan__dlm
WHERE ssot__StartDateTime__c >= TIMESTAMP '2026-08-04 00:00:00'
      AND ssot__StartDateTime__c <  TIMESTAMP '2026-08-11 00:00:00'
GROUP BY ssot__OperationName__c
ORDER BY p95_ns DESC
```

How to read the result: Compare the same operation across two runs. A p95 that improved while p50 stayed flat means you fixed the tail — usually the right outcome. p50 down and p95 unchanged means you made the common case faster and left the bad case alone, which users experience as no change at all.

Pitfall. Spans carry no agent identity, so this is org-wide. If several agents changed in the same window you cannot separate them here — use C5/C6/C7, which can be agent-scoped, as the corroborating view.

R3. Quality distribution, two windows

DERIVED · **Answers:** whether the score mix shifted after a change.

Run it when: you tightened instructions and want to know if graded quality actually moved.

```
SELECT t.ssot__Value__c AS quality_score,
       COUNT(*) AS graded
FROM ssot__AiAgentTagAssociation__dlm ta
JOIN ssot__AiAgentTag__dlm t
  ON t.ssot__Id__c = ta.ssot__AiAgentTagId__c
WHERE ta.ssot__AiAgentSessionId__c IS NOT NULL
      AND ta.ssot__CreatedDate__c >= '<WINDOW_START_ISO>'
      AND ta.ssot__CreatedDate__c < '<WINDOW_END_ISO>'
GROUP BY t.ssot__Value__c
ORDER BY t.ssot__Value__c
```

How to read the result: Compare the shape, not the average. Moving sessions out of the 1–2 bucket matters far more than nudging the mean, and an average can improve while the worst cases get worse. Grouping rather than averaging also sidesteps the string-column cast entirely — this is how the shipped Apex service does it.

Pitfall. Tag associations are written asynchronously, so a window that ends too recently under-counts. Leave at least a day of margin at the end.

R4. Daily trend for one subagent

DERIVED · **Answers:** turn volume and latency per day for a single topic.

Run it when: you want to see when a regression started rather than that it exists.

```
SELECT SUBSTRING(i.ssot__StartTimestamp__c, 1, 10) AS day,
       COUNT(*) AS turns,
       AVG(datediff('millisecond',
                    TRY_CAST(i.ssot__StartTimestamp__c AS TIMESTAMP),
                    TRY_CAST(i.ssot__EndTimestamp__c AS TIMESTAMP))) AS avg_ms
FROM ssot__AiAgentInteraction__dlm i
JOIN ssot__AiAgentSessionParticipant__dlm p
  ON p.ssot__AiAgentSessionId__c = i.ssot__AiAgentSessionId__c
 AND p.ssot__AiAgentSessionParticipantRole__c = 'AGENT'
 AND p.ssot__AiAgentApiName__c = '<AGENT_API_NAME>'
WHERE i.ssot__AiAgentInteractionType__c = 'TURN'
      AND i.ssot__TopicApiName__c = '<TOPIC_API_NAME>'
      AND i.ssot__StartTimestamp__c >= '<ISO_CUTOFF>'
      AND i.ssot__EndTimestamp__c IS NOT NULL
GROUP BY SUBSTRING(i.ssot__StartTimestamp__c, 1, 10)
ORDER BY day ASC
```

How to read the result: The day the average steps up is the day something changed — line it up against your deploy history. Because the timestamp is a string, SUBSTRING(..., 1, 10) gives you the UTC date directly with no casting; this is one place the string typing is convenient.

Pitfall. UTC days, not local. For an agent serving one region the boundary will not line up with the business day — shift the substring window or accept the offset.

Group P — Scenario playbooks: chain the queries

Real debugging is a sequence, not one query. Each playbook is a numbered chain using the recipe ids above. Start with P0 if the session is recent — it is the only one that works before the STDM hierarchy materializes.

P0 · 'This just happened and I need to know now'

- 1. 0d — check the session's age. Started in the last few hours? Everything below Group B will be empty, and that is expected.
- 2. A2 / D1 — the gateway rows are already there. You get every model call, its prompt template, its token counts and its timestamps.
- 3. D6(b) — read `prompt__c` on the planner calls. You can see what the agent was told, what tools it was offered, and infer the routing from `promptTemplateDevName__c`, before a single Step row exists.
- 4. X2 — the ToolCall metadata shows which actions it tried to call, with arguments.
- 5. D2 / D10 — token shape of the conversation, if cost is the complaint.
- 6. Tomorrow: rerun B3 / B4 / C3 for the step-level trace and exact ordering.

Verdict: you can diagnose prompt content, model choice, cost, and attempted tool calls immediately. Only step ordering, step errors and step timing have to wait. If someone tells you tracing is broken on a session from this morning, this is the playbook that shows it is not.

P1 · 'This turn was slow'

- 1. C1 — rank the turns; grab the slowest interaction id. Then **C2**, because a turn that never ended will not appear in C1 at all.
- 2. C3 — break that turn into per-step durations; note the worst step type.
- 3. T2 — pull the trace id (from `ssot__AttributeText__c`, not the `TraceId` column).
- 4. T3 — every span for the trace, longest first, in exact nanoseconds. **T8** if the children do not add up to the root.
- 5. T6 — is this a planner problem, an action problem, or a retrieval problem?

Verdict: top span `run.llmstep` → model latency, go to D9/D14 for prompt size. Top span `run.action.*` → slow backend action, go to C7 to see if it is slow everywhere. Large unaccounted time in T8 → the turn is asking the platform to coordinate too much; reduce step count.

P2 · 'This conversation was expensive'

- 1. D1 — the raw ledger for the session.
- 2. D2 — group tokens per turn to find the priciest `generationGroupId__c`.
- 3. D10 — is prompt size climbing turn over turn? That is history accumulation, and it is the most common cause.
- 4. D11 — several near-identical calls in one group? That is retries, and you paid for each.
- 5. D9 — for the guilty template, is the weight in the input or the output?
- 6. D13 — check `numGenerations__c` and `temperature__c` before blaming the prompt.

Verdict: climbing prompt tokens → trim conversation history in the agent's context config. Repeated near-identical calls → planner retries, simplify the instructions that confuse it. High input-to-output ratio → trim system instructions and tool definitions. `numGenerations__c > 1` → you are paying N times per call for no reason.

P3 · 'The agent gave a wrong answer'

- 1. B3 — did it route to the expected subagent? A null `ssot__TopicApiName__c` is an unrouted turn; B8 shows whether it is a pattern.
- 2. B4 — open the turn's steps; check the ordering.

- **3. X1** — read the `ACTION_STEP` input and output. This is where most wrong answers actually come from: the action succeeded, with the wrong arguments.
- **4. X3** — diff the variable snapshots to find the step where expected context went missing.
- **5. D6** — for the turn's **LLM_STEP** specifically, read the exact prompt and response.
- **6. E2** — did the guardrail flag LOW adherence? **Q4** — what did `TaskResolution` score?
- **7. Q6** — did the user actually complain? **X4** — did they have to ask twice?

Verdict: wrong routing → sharpen topic descriptions. Wrong action arguments (X1) → fix the action's input schema or the instructions describing when to call it. Missing context in the prompt → fix grounding. LOW adherence with a correct prompt → the instructions are being ignored, not missing; tighten and add examples. *Note the order: step 3 before step 5. Sending an arbitrary step id to D6 returns nothing for ACTION_STEP rows, which is where wrong answers usually live.*

P4 · 'Something is failing'

- **1. E1** — errored `ACTION_STEP` rows only, with all four non-error values excluded. Every row is real.
- **2. B4** — open that turn's full step sequence for context around the failure.
- **3. T2 → E3** — the error spans for that specific trace, with `ssot__TelemetrySpanAttributeText__c` for the stack.
- **4. T4** — is this failure happening across the fleet, or just here?
- **5. X1** — what was the action called with when it failed?

Verdict: the span attribute text usually names the failing service or exception. If E1 is empty but users report failures, the failure is not an exception — it is a wrong answer, so go to P3 instead.

P5 · 'Is my knowledge base the problem?'

- **1. Q7** — sort RAG metrics by faithfulness ascending; below 0.8 is hallucination risk.
- **2. Q4** — what did the `TaskResolution` and `InstructionAdherence` detectors say for the same generations?
- **3. D6** — for a low-scoring utterance, read the actual prompt to see what was retrieved and handed to the model.
- **4. X4 / Q6** — did the user ask again, or thumbs-down it?

Verdict: low context precision → the retriever is pulling junk; fix chunking, filters or the source. Low faithfulness with good precision → the model ignored good context; fix instructions to force grounding. Good scores but unhappy users → it is not the knowledge base, go to P3.

P6 · 'Did my change actually help?'

- **1. 0b** — get both version api names and confirm both have traffic.
- **2. R1** — tokens per turn by version. Volume-independent, so adoption changes do not fool you.
- **3. R2** — p50 and p95 latency across the two windows.
- **4. R3** — quality score distribution, comparing the shape not the mean.
- **5. R4** — daily trend for the topic you changed, to pin the date.
- **6. B8** — did the unrouted rate move?

Verdict: improve tokens-per-turn and the 1–2 quality bucket together and the change was real. If the average improved but the worst bucket grew, you optimised the easy cases and made the hard ones worse — which is the failure mode averages are best at hiding.

Reading guide — trace → cause → fix

Once you have the trace in front of you, this maps what you see to the likely cause and where to change it.

What you see	Likely cause	Where to fix it
One LLM_STEP / run.llmstep span dominates	Model latency or an oversized prompt	Trim prompt and context (D9, D14); consider a faster model (D5)
An ACTION_STEP / run.action.* span dominates	Slow backend action — Apex, Flow, external callout	Rank it fleet-wide with C7; optimise the action, add caching or a timeout
Many LLM_STEP rows in one turn	Planner retries or over-decomposition	Confirm with D11; simplify topic and action instructions
Prompt tokens climbing turn over turn (D10)	Conversation history replayed in full into every prompt	Trim history in the agent's context configuration — not in the prompt text
Root span far exceeds the sum of its children (T8)	Queueing / coordination overhead, not any one operation	Reduce step count per turn (C4); fewer, larger actions
ACTION_STEP error that is none of the four non-error values	A step genuinely threw	Read the message; open the error span (E3) for the stack
Guardrail 'value': 'LOW'	Model strayed from its instructions	Tighten instructions, add examples; check the prompt in D6
Null or NOT_SET TopicApiName__c (B8)	Planner could not decide which subagent owned the utterance	Sharpen and de-overlap subagent descriptions
Action succeeded, answer still wrong (X1)	Right action, wrong arguments	Fix the action's input schema, or the instructions on when to call it
Expected variable empty mid-turn (X3)	An earlier step failed to set state	Fix the step that should have populated it, not the one that failed
Low RAG faithfulness (Q7)	Answer not grounded in retrieved context	Force grounding in instructions; verify the retriever
Low RAG context precision (Q7)	Retriever returning irrelevant chunks	Fix chunking, filters, or the knowledge source
Low TaskResolution (Q4) with healthy traces	The agent ran perfectly and still did not solve the problem	Capability gap — the agent needs an action or knowledge it does not have
User repeated themselves (X4)	The previous answer missed	Read that turn with P3; this is unprompted ground truth

Investigation order, when you do not know where to start

- **Scope it** — 0a data space, 0b agent name, 0c the join, 0d session age. Most "broken" queries are unscoped ones.
- **Check freshness before ingestion** — A1/A2/A3. Gateway present but Interaction absent means *wait*, not *fix*. Go to P0.
- **Read the conversation** — B1 to find the session, B2 for the transcript, B3 for routing.
- **Localise** — slow? Group C, then T. Expensive? Group D. Broken? Group E. Wrong? Group X, then Q.
- **Get exact numbers** — Group T spans for timing, D6 for the literal prompt and response, X1 for action arguments.
- **Judge quality** — Q4 trust-layer detectors and Q6 user feedback need no add-on. Q1/Q2/Q3 need Agent Optimization. Q7 needs RAG actions.

- **Prove the fix** — Group R and P6. A change you cannot measure is a change you cannot defend.

Tip. Every <SESSION_ID>, <INTERACTION_ID>, <TRACE_ID>, <STEP_ID>, <AGENT_API_NAME> and <ISO_CUTOFF> is a placeholder — replace it with a real value from an earlier query. Time windows are examples; widen or narrow them to your case.

Troubleshooting — when the queries will not run

Ordered by how often each one is the actual cause. Work down the list; the top four account for most of it, and none of them require changing any configuration.

First, separate the two failure modes. "It ran and gave me nothing" and "it would not run" have completely different causes. Zero rows is almost always scope or timing (T1–T5). An error message is almost always permissions, a name, or a function (T6–T10). Do not go changing org settings because a query returned zero rows.

It ran, but returned nothing

#	Symptom	Most likely cause and fix
T1	Every query returns zero rows, including A1	Wrong data space. The default is often not the one your agent writes to. Run 0a , get the real name, and select it in the editor or pass it to the API. This is the single most common cause and it looks exactly like broken ingestion.
T2	A1/A2 return rows, but B3/B4/C-group are empty for a recent session	Nothing is wrong — the data has not landed. Session, Participant and Gateway materialize in minutes; Interaction, Step, Message, Generation and Moment take hours to days. Confirm the session's age with 0d , then debug from the gateway side using playbook P0 . Come back tomorrow for the step trace.
T3	Nothing at all for a session you know happened	Tracing was enabled after the conversation. Session Tracing is not retroactive — it only captures conversations that start after enablement. Check when it was switched on before assuming a bug.
T4	A gateway query on a known session id returns nothing	The quoted sessionId__c. The stored value includes literal double-quotes, so a bare equals on the UUID matches nothing. Use LIKE '%<sid>%' or = '<sid>' (Rule 4).
T5	A fleet-wide query returns far fewer rows than expected	Your own filters. In order: the agent API name in the Participant join (is it the API name or the display label?), feature__c = 'plannerservice', sessionId__c != '"no_session"', and any IS NOT NULL guard on an end timestamp. Comment them out one at a time.
T6	An outcome or status filter returns nothing	Enum values differ by org. Session end types especially — the references disagree with each other, and the field is absent from the live-verified enum list. Run B6 first and filter on what your org actually records. Span sso__StatusCode__c has no documented enum at all, so check it the same way.
T7	The T group is empty but everything else works	Agent Platform Tracing is off, or the collection cycle has not run. Needs enableAgentforcePlatformTracing, API v68.0+, and the provisioned PlatformObservability permission. Collection runs every five minutes, so a very recent turn may not have spans yet.
T8	Q1/Q2/Q3 return nothing but the rest of Q works	Agent Optimization is not enabled. The Moment and tag DMOs need the add-on. Q4, Q5 and Q6 do not — use those for quality signal in the meantime.

It would not run at all

#	Error you see	Most likely cause and fix
T9	No Data Cloud app; no Query Editor tab	Missing permission set or no Data Cloud. You need Data Cloud User on your user. If the app itself is absent, Data Cloud is not provisioned on the org — that is a licensing conversation, not a settings one.
T10	"Object not found" / table does not exist	In order: (a) casing — Session, Interaction and Step use uppercase AI Agent, everything else lowercase Ai Agent, and the references genuinely disagree, so try both; (b) data space — a DMO can exist in one space and not another; (c) not provisioned — the Optimization DMOs need the add-on, and the RAG metrics DMO only exists once the agent actually uses knowledge retrieval. Settle it with <code>sf data360 query describe --table <name></code> .
T11	"Column does not exist"	Check the <code>ssot__</code> prefix. It is not universal: the GenAI* family has no prefix at all (fields end in plain <code>__c</code>), and a few Message columns (<code>Modality__c</code> , <code>MessageStartTime__c</code> , <code>MessageEndTime__c</code>) drop it too.
T12	A function is rejected — <code>datediff</code> , <code>TRY_CAST</code> , <code>PERCENTILE_CONT</code>	All three are documented in the Data 360 SQL reference, so a rejection means your engine spells it differently or the exact syntax form is off — most often <code>PERCENTILE_CONT . . . WITHIN GROUP</code> . Fallbacks that work everywhere: for durations, select the raw start and end timestamps and subtract outside SQL; for percentiles, use <code>AVG</code> and <code>MAX</code> , or order by duration and read the row at the 95th position. This is exactly why those recipes are labelled UNCONFIRMED.
T13	A comparison fails on a timestamp column	You are almost certainly comparing a string column to a date expression. Use the ISO literal (Rule 1). If you must cast, cast the expression rather than the column: a bare column keeps whatever indexing or partitioning the engine has on it, and a computed one does not.
T14	Query times out, or the result set is too large	Add a <code>LIMIT</code> , narrow the window, or move to <code>sqlv2</code> (paginated) or <code>async-create</code> (large exports). On the Query API, remember the one-hour total and three-minute inter-batch limits.
T15	Apex query <code>AnsSqlv2</code> returns null or throws	The data-space argument is required and is not defaulted. Wrap in <code>try/catch</code> and read the exception message — that is where the real SQL error text is; a null return on its own tells you nothing.
T16	Results look right but the columns are shifted	Rows come back positionally in both the API and Apex. Use the response metadata block's <code>placeInOrder</code> instead of assuming your <code>SELECT</code> order was preserved.

The results are wrong rather than absent

- **Every error looks like a failure.** Four values mean "no error" — `NOT_SET`, `UNSET_VALUE`, the empty string, and the literal `None` that guardrail steps carry. Filter all four, and restrict to `ACTION_STEP` (E1).
- **Durations look absurd.** Span `ssot__DurationNumber__c` is nanoseconds — divide by `1e6` for milliseconds (Rule 6).
- **The slowest turn is missing from the ranking.** A turn that never ended has a null end timestamp and drops out of every duration sort. That is what C2 is for.
- **Token totals are higher than the agent's traffic.** The gateway table holds every LLM call in the org, including Prompt Builder previews and test runs. Exclude `sessionId__c = 'no_session'` and filter to `feature__c = 'plannerservice'`.
- **A field parses as garbage.** Several are HTML-escaped, and `LLM_STEP` and guardrail input/output are Python dict strings, not JSON. Unescape first; only `ACTION_STEP` input/output is real JSON.

- **Averages across a multi-agent org.** If you dropped the Participant join, you are averaging every agent together. Group 0 exists for this.

Still stuck? Reduce to the smallest thing that should work: `SELECT COUNT ( * ) FROM ssot__AIAgentSession__d lm`. If that fails, it is access, data space, or provisioning. If it succeeds, add one clause at a time back onto your real query — the clause that zeroes it out is your answer, and it is usually a filter rather than the platform.

Further reading

Where to go when this notebook does not cover it. The Salesforce Help pages describe the data model and the setup; the Developer pages describe the SQL dialect and the APIs. Every URL below is a live link.

Data model reference — what every DMO and field means

Page	What it covers	Link
Session Tracing Data Model	Session, Participant, Interaction, Message and Step — the backbone of Groups A through E.	help.salesforce.com/.../generative_ai_session_trace_data_model
Generative AI Audit and Feedback Data Model	The 13-DMO GenAI* family: gateway requests and responses, generations, content quality and category, feedback, tool-call metadata. Groups D, Q and X.	help.salesforce.com/.../generative_ai_feedback_data_model
Agent Optimization Data Model	The six DMOs behind Q1, Q2, Q3 and R3: Moment, MomentInteraction, Tag, TagDefinition, TagDefinitionAssociation and TagAssociation.	help.salesforce.com/.../generative_ai_optimize_data_model
Agent Platform Tracing	ssot__TelemetryTraceSpan__d__lm, operation and service names, and how to switch tracing on. The whole T group.	help.salesforce.com/.../generative_ai_platform_trace
Agentforce Observability Setup and Access	The setup walkthrough: which toggles, which permission sets, which licences — and the warning that tracing is not retroactive.	help.salesforce.com/.../articleView?id=005237036

Trust the describe over the doc. The Help pages list *logical* API names and aspirational enum values that diverge from what the API actually exposes — table casing and session end types are both known examples. When a Help page and your org disagree, the org is right. Describe the table and move on.

SQL dialect — what you can actually write

Page	What it covers	Link
Data 360 SQL Reference	The dialect index — start here.	developer.salesforce.com/.../references/dc-sql-reference/
Date/Time Functions and Operators	current_date, datediff, date_add, interval literals. Behind Rules 1 and 2, and all of Groups C and T.	.../dc-sql-reference/datetime-func.html
Date/Time Types	Which date and time types exist, and how interval literals are written.	.../dc-sql-reference/datetime.html
Type Conversions	CAST, :: and TRY_CAST — the difference being that TRY_CAST returns null instead of throwing.	.../dc-sql-reference/type-conversion.html
Aggregate Functions	Including the ordered-set aggregates PERCENTILE_CONT and PERCENTILE_DISC used in T7, D14 and R2.	.../dc-sql-reference/aggregate.html
Window Functions	For running totals and per-group ranking beyond what this notebook uses.	.../dc-sql-reference/window.html

Running the queries

Page	What it covers	Link
Query Data in Data 360	The overview of every execution path: editor, APIs, JDBC, Python connector, Apex.	developer.salesforce.com/...query-guide-get-started.html
Request Data With Query V2	The POST /api/v2/query contract — headers, body, nextBatchId pagination, and the one-hour and three-minute limits. Path B.	.../data-cloud-query-api-reference/c360a-api-query-v2.html
Data 360 SQL Query APIs	The other client options — JDBC, the Python connector, Power BI, Apex.	.../guide/dc-sql-query-apis.html
Export Agentforce Session Tracing Data (Beta)	The OpenTelemetry export API, if you would rather ship traces to your own observability stack than query them here. Beta: one session at a time, and roughly 72 hours of retention.	developer.salesforce.com/docs/ai/agentforce/guide/otel-api.html

Developer links shown as .../ continue from `developer.salesforce.com/docs/data/data-cloud-query-guide/`. Note the October 2025 rebrand: "Data Cloud" and "Data 360" are the same product, and both names appear across the documentation with no functional difference.

Appendix — Conflicts to settle in your own org

Six places where the reference documentation genuinely disagrees with itself. Each is cheap to resolve with one query against your own org. Do it once, write the answer down, and stop guessing.

#	The disagreement	How to settle it
1	Table casing. The live v66.0 describes give ssot__AIAgentSession__d__l__m (uppercase AI); the STDM schema reference and the shipped Apex both use ssot__AiAgentSession__d__l__m.	<code>SELECT ssot__Id__c FROM <each casing> LIMIT 1.</code> This notebook uses uppercase for Session / Interaction / Step.
2	Session end types. Five sources, five different value lists. The field is absent from the live-verified enum table.	B6. Run it before writing any outcome filter. Expect a large null bucket.
3	<code>Step.ssot__GenAiGatewayRequestId__c</code> . One source calls it schema-only and frequently <code>NOT_SET</code> ; another calls it non-null on every <code>LLM_STEP</code> and uses it as a join key in shipped code.	Count it: <code>SELECT COUNT(*), SUM(CASE WHEN ssot__GenAiGatewayRequestId__c = 'NOT_SET' THEN 1 ELSE 0 END) ... WHERE step type = 'LLM_STEP'</code> . If it is mostly populated, D8's per-subagent attribution becomes viable.
4	Error field null semantics. One source says never <code>NULL</code> , always <code>NOT_SET</code> ; another says null when there is no error.	E1 handles both <code>(NOT IN (...) AND IS NOT NULL)</code> . No action needed unless you write your own filter.
5	RAG metric join key. <code>ssot__AiGatewayRequestId__c</code> versus <code>RetrieverTraceId__c</code> ; the DMO is absent from the v66.0 describe reference entirely.	Describe the DMO in your org and see which column exists. Until then Q7 scopes by score threshold only.
6	Message DMO name. One reference names it <code>ssot__AiAgentSessionMessage__d__l__m</code> in one table and <code>ssot__AiAgentInteractionMessage__d__l__m</code> in another.	Two other sources corroborate <code>InteractionMessage</code> , which is what this notebook uses. Do not "correct" it.

Sources: live Data Cloud v66.0 object describes (session, interaction, step, message, participant, moment and tag DMOs; `GenAiGatewayRequest`, `GenAiGeneration`, `GenAiContentQuality`, `GenAiContentCategory`, `GenAiFeedback`, `GenAiGtwyRequestMetadata`; `TelemetryTraceSpan`); the 26 executable query templates and pipeline contract in `agentforce-d360-analyze`; the STDM schema and query references and the shipped `AgentforceOptimizeService` Apex class in `agentforce-observe`; and the Salesforce Data 360 SQL Reference for date/time functions, type conversion and aggregates. VERIFIED = names and query shape confirmed against something that runs. DERIVED = our aggregation or duration math on real fields. UNCONFIRMED = plausible, never run live. Verify against your own org before trusting fleet-wide roll-ups.